Memoria Técnica

cv-lit

Sistema de Monitorización Automática

de Línea de Costa

Guardamar del Segura · Alicante · España

Tech4D Lab
Universidad de Alicante
`nickhernd`

Junio 2026

Resumen

cv-lit es un sistema software de monitorización automática de línea de costa mediante visión por computador, desarrollado para el Ayuntamiento de Guardamar del Segura en colaboración con el Instituto de Ecología Litoral (IEL) y la Universidad de Alicante.

El sistema procesa imágenes oblicuas de **6 cámaras fijas Obscape** instaladas a lo largo del frente litoral (tramo norte–sur $\approx$ 3,5 km), extrayendo automáticamente la línea de costa y proyectándola a coordenadas geográficas UTM EPSG:25830 (ETRS89 / UTM zona 30N).

Esta memoria recoge la arquitectura completa del sistema, los algoritmos de procesamiento de imagen (SIFT, RANSAC, SAM), el módulo de alineación masiva no-destructiva diseñado en la iteración de junio de 2026, la interfaz web de validación por lotes, los resultados de calibración geométrica obtenidos para las seis cámaras, y la documentación operacional completa para el despliegue y mantenimiento del sistema.

Índice general

1. Descripción del Proyecto **7**
- 1.1. Contexto y motivación . . . 7
- 1.2. Partes implicadas . . . 7
- 1.3. Objetivos del sistema . . . 7
- 1.4. Restricciones no negociables . . . 8

2. Infraestructura de Cámaras **9**
- 2.1. Sistema Obscape . . . 9
- 2.2. Distribución geográfica . . . 9
- 2.3. Descarga automática de imágenes . . . 9
  - 2.3.1. Automatización con Cron . . . 10

3. Arquitectura del Sistema **11**
- 3.1. Stack tecnológico . . . 11
- 3.2. Estructura de directorios . . . 12
- 3.3. Pipeline de procesamiento . . . 13

4. Calibración Geométrica **14**
- 4.1. ¿Qué es la calibración y para qué sirve? . . . 14
- 4.2. De dónde salen los puntos de control (GCPs) . . . 15
- 4.3. Flujo actual: calibración interactiva desde la interfaz web . . . 15
- 4.4. Estimación con RANSAC . . . 16
  - 4.4.1. Aviso de geometría inestable . . . 17
- 4.5. Dos umbrales que no hay que confundir . . . 17
- 4.6. Métricas del ajuste: RMSE y MAE . . . 18
- 4.7. Resultados de calibración (campaña junio 2026) . . . 18
- 4.8. Imágenes de diagnóstico . . . 19
- 4.9. Comparación con la homografía de alineación (capítulo 5) . . . 21

5. Módulo de Alineación Masiva **22**
- 5.1. Motivación . . . 22
- 5.2. Arquitectura de estados . . . 22
  - 5.2.1. Estructura del directorio temporal por job . . . 23
- 5.3. Pipeline de alineación: algoritmo . . . 23
  - 5.3.1. Parámetros configurables . . . 23
  - 5.3.2. Estrategia de dos pasadas . . . 24
  - 5.3.3. Flujo completo del pipeline . . . 24
- 5.4. Renderizado del mapa de delta . . . 25
  - 5.4.1. Capa 1 — Heatmap de diferencia absoluta . . . 25

5.4.2. Capa 2 — Divergencia de bordes (overlay cian) . . . 25
5.4.3. Capa 3 — Métrica escalar δ_{px} (flujo óptico Farneback) . . . 25
5.5. API REST del módulo . . . 26
5.6. Modelos de dominio . . . 26
5.7. Flujo de commit (two-phase write) . . . 27

6. Máscaras de Zonas Estables SIFT **28**
6.1. Problema: objetos dinámicos en SIFT . . . 28
6.2. Solución: máscaras de regiones estables . . . 28
6.3. Zonas por cámara . . . 29
6.4. Cómo ajustar una máscara . . . 29

7. Interfaz Web **31**
7.1. Arquitectura frontend . . . 31
7.2. Componente BatchAlignment.vue . . . 31
7.2.1. Variables de estado . . . 31
7.2.2. Props y eventos . . . 32
7.2.3. Layout de la fase reviewing . . . 32
7.2.4. Indicadores del filmstrip . . . 33
7.2.5. Polling de progreso . . . 33
7.3. Flujo completo de uso . . . 33

8. Región de Interés (ROI) **34**
8.1. Criterios de delimitación . . . 34
8.2. Coordenadas por cámara . . . 34

9. Segmentación y Extracción de Línea de Costa **36**
9.1. Segmentación semántica con SAM . . . 36
9.2. Extracción de línea de costa . . . 37
9.3. Proyección a EPSG:25830 . . . 37
9.4. Exportación GeoJSON . . . 37
9.5. Índice de confianza (`Confianza_IA`) y una limitación conocida . . . 38

10. Validación y Métricas de Calidad **40**
10.1. Estrategia de validación . . . 40
10.2. Métricas . . . 40
10.3. Criterios de aceptación . . . 41
10.4. Factores de error a monitorizar . . . 41
10.5. Validación en QGIS . . . 41

11. Módulo de Automatización (Auto Mode) **42**
11.1. ¿Qué problema resuelve? . . . 42
11.2. Requisito previo: la cámara debe estar ya calibrada . . . 42
11.3. Las cuatro fases . . . 43
11.4. Por qué el commit automático es seguro sin revisión humana . . . 44
11.5. Modelos de dominio . . . 44
11.6. API REST del módulo . . . 45
11.7. Nota de empaquetado: import dinámico y PyInstaller . . . 45

12. Estado del Proyecto y Roadmap **46**
12.1. Estado actual (agosto 2026) . . . 46
12.2. Completado en la iteración de junio 2026 . . . 46
12.3. Fase 3 — Automatización y robustez (completada) . . . 47
12.4. Fase 4 — Entrega e integración (completada) . . . 47
12.5. Ronda de pulido para producción (julio–agosto 2026) . . . 47
12.6. Ronda de refinamiento — 31 de agosto de 2026 . . . 48
12.6.1. La confianza baja tenía tres causas independientes, no una . . . 48
12.6.2. Un cuarto problema, específico de cámara 6: un punto de calibración con `utm=[0,0]` . . . 48
12.6.3. Reequilibrio de `confidence_index()` (continuación de §9.5) . . . 49
12.6.4. Otros arreglos reales de esta ronda . . . 49
12.6.5. Estado final de calibración de las 6 cámaras (verificado en el `.exe` real) . . . 50
12.7. Ronda de funcionalidades nuevas — reunión de equipo, 11 de septiembre de 2026 . . . 50

13. Guía de Operación **52**
13.1. Abrir la aplicación . . . 52
13.2. Recorrido por pantallas . . . 53
13.2.1. Dashboard . . . 53
13.2.2. Carga de imágenes . . . 53
13.2.3. Resultados . . . 54
13.2.4. Mapa GeoJSON . . . 56
13.2.5. Calibración . . . 56
13.2.6. Cámaras . . . 57
13.2.7. Modo automático . . . 58
13.2.8. Configuración . . . 58
13.3. Flujo de trabajo recomendado (día a día) . . . 59
13.4. Funcionalidades en pausa . . . 59
13.4.1. Transectos — anchura de playa . . . 60
13.4.2. Entrenamiento línea de costa — sección temporal . . . 60
13.4.3. Lo que queda pendiente, a propósito . . . 62

14. Instalación y Configuración **63**
14.1. Requisitos del sistema . . . 63
14.2. Instalación del entorno Python . . . 63
14.3. Instalación del frontend . . . 64
14.4. Configuración de la API Obscape . . . 64
14.5. Generar el instalador de Windows . . . 64
14.6. Entorno de desarrollo y herramientas de línea de comandos . . . 66
14.6.1. Entrenamiento de un modelo propio (YOLO-seg) . . . 68

A. Convenciones de código y Git **69**
A.1. Estrategia de ramas . . . 69
A.2. Nomenclatura de ficheros . . . 69
A.3. Proyección obligatoria . . . 69

B. Glosario **70**

C. Historial de cambios **72**

Índice de figuras

3.1. Pipeline completo de procesamiento cv-lit 13

4.1. Los tres pasos del asistente de calibración implicados en este capítulo . 15
4.2. Diagnóstico de calibración — reproyección de GCPs (vectores ×10), campaña de junio de 2026 20

5.1. Máquina de estados del módulo BatchAlignment 22
5.2. Flujo de commit no-destructivo 27

11.1. Pipeline de Auto Mode — cada fase reutiliza código ya existente, sin reimplementar nada 43

13.1. Dashboard, con las 6 cámaras calibradas y el botón *Informe PDF* sustituyendo al antiguo *Imprimir Reporte*. 53
13.2. Carga de imágenes — selector de cámara, zona de subida, filtro de fechas y perfil de calibración cargado. 54
13.3. Resultados — selector de cámara/imagen, panel de métricas y visor de la línea de costa. 55
13.4. Calibración — vista general con el estado real de las 6 cámaras: RMSE, MAE, GCPs e inliers. 57
13.5. Cámaras — estado de las 6 estaciones (RMSE, GCPs, fecha de calibración). 57
13.6. Modo automático — panel de procesamiento por rango de fechas y estado de RMSE por cámara. 58
13.7. Configuración — credenciales de Obscape, ubicación de los datos y borrado de imágenes/resultados. La tarjeta de Entrenamiento que mostraba la ruta de esa sección (§13.4) está oculta con el mismo criterio. 59

Índice de cuadros

1.1. Actores del proyecto 7
1.2. Atributos del GeoJSON de salida 8

2.1. Estaciones Obscape en Guardamar del Segura 9

3.1. Dependencias principales 11

4.1. Umbral por varilla (píxeles) frente a umbral de aceptación (metros) . . 17
4.2. Resultados de precisión por cámara — campaña de calibración inicial . 18
4.3. Las dos homografías del sistema, lado a lado 21

5.1. Parámetros del pipeline de alineación (`batch_alignment.py`) 23
5.2. Leyenda del heatmap HOT 25
5.3. Endpoints del router `/api/batch/` 26

6.1. Configuración de máscaras SIFT por cámara 29

7.1. Props del componente BatchAlignment 32
7.2. Código de color en el filmstrip 33

8.1. ROIs de producción por cámara (calibration/roi_config.json) 34

10.1. Umbrales de calidad del sistema 41

11.1. Endpoints del router `/api/automode/` 45

12.1. Estado de hitos del proyecto 46
12.2. RMSE tras la ronda de refinamiento del 31 de agosto de 2026 50

14.1. Requisitos hardware y software 63
14.2. Parámetros de conexión Obscape 64

C.1. Historial de versiones del sistema 72

Capítulo 1

Descripción del Proyecto

1.1 Contexto y motivación

La playa de Guardamar del Segura es un espacio de alto valor ecológico y turístico sujeto a procesos de erosión y acreción que requieren seguimiento sistemático. La medición manual de la línea de costa mediante levantamientos GNSS es costosa, puntual y no escala para cubrir todo el frente litoral con frecuencia horaria.

El proyecto **cv-lit** surge para automatizar esta medición aprovechando la infraestructura de cámaras fijas ya desplegada por el servicio de vigilancia de la playa, convirtiendo cada fotografía en un dato georreferenciado de la posición de la interfaz arena–mar.

1.2 Partes implicadas

Cuadro 1.1: Actores del proyecto

Rol	**Entidad**
Cliente	Ayuntamiento de Guardamar del Segura
Datos GNSS (GCPs)	Instituto de Ecología Litoral (IEL)
Desarrollo	1 ingeniero a tiempo parcial (Tech4D Lab, UA)
Supervisión	Universidad de Alicante
Duración	6 meses (enero – junio 2026)

1.3 Objetivos del sistema

El producto final es un **GeoJSON proyectado en EPSG:25830** con los siguientes atributos obligatorios por cada detección:

Cuadro 1.2: Atributos del GeoJSON de salida

Campo	Tipo	Descripción
`ID_Camara`	String (`ÇAM_1"..ÇAM_6"`)	Identificador de cámara origen
`Timestamp`	ISO 8601 UTC	Fecha y hora de captura
`Confianza_IA`	Float (0–1)	Score de confianza del modelo SAM
`Area_Seca_m2`	Float	Superficie de playa seca (m^2)
`RMSE_m`	Float	RMSE de la calibración activa en el momento del análisis
`n_puntos`	Integer	Número de puntos de la polilínea de costa

1.4 Restricciones no negociables

- Toda implementación **debe** producir GeoJSON en **EPSG:25830**.
- Fase horaria prioritaria: **12:00h solar local**.
- Modelo de segmentación: **SAM** (Segment Anything Model, Meta AI).
- Cada cámara tiene su propio perfil de calibración *independiente*.
- Criterio de aceptación de precisión: **RMSE $<$ 2 m** en condiciones estándar (ver `RMSE_GOOD_M`, capítulo 4).

Capítulo 2

Infraestructura de Cámaras

2.1 Sistema Obscape

Las seis cámaras están gestionadas por la plataforma Obscape y se acceden via API REST autenticada. La tabla 2.1 lista los parámetros de registro de cada estación.

Cuadro 2.1: Estaciones Obscape en Guardamar del Segura

Cámara	Station	Referencia	Latitud (°N)	Longitud (°E)	Última act.
CAM 1	8213	PTM61471	38.110327	−0.643027	20-05-2026
CAM 2	8214	PTM61474	38.096291	−0.645725	20-05-2026
CAM 3	8212	PTM61473	38.087348	−0.646994	20-05-2026
CAM 4	8211	PTM61475	38.087385	−0.647078	20-05-2026
CAM 5	8209	PTM61472	38.076774	−0.648117	20-05-2026
CAM 6	8210	PTM61470	38.076796	−0.648114	20-05-2026

Nota geográfica

CAM 3 y CAM 4 son casi coincidentes ($\Delta \approx 8$ m) y cubren el mismo tramo desde ángulos ligeramente distintos. Lo mismo ocurre con CAM 5 y CAM 6. Esto permite comparación cruzada de la línea detectada para validación interna.

2.2 Distribución geográfica

Las cámaras se distribuyen de norte a sur a lo largo de $\approx 3{,}5$ km de frente litoral. La cobertura solapada garantiza continuidad en la extracción de línea de costa sin lagunas.

2.3 Descarga automática de imágenes

El módulo `acces_api/scheduled_download.py` implementa una **ventana deslizante de 14 días** que garantiza recuperación automática ante fallos puntuales de descarga:

Descarga estándar

```
# Últimas 2 semanas, hora solar 12:00 (por defecto)
python3 acces_api/scheduled_download.py

# Personalizar ventana y hora
python3 acces_api/scheduled_download.py --days 30 --hour 15

# Descarga masiva histórica (todas las horas)
python3 acces_api/scheduled_download.py --days 90 --hour all
```

El sistema evita duplicados comprobando la existencia del fichero antes de descargar y utiliza un nombre determinista: `UNIX_YYYYMMDD_HHMMSS_REF.jpg`.

2.3.1 Automatización con Cron

Para descarga diaria autónoma:

/etc/cron.d/cvlit-download

```
# Descarga diaria a las 13:00h (12h solar + margen de publicación API)
0 13 * * * cd /home/nickhernd/Desktop/cv-lit && \
  python3 acces_api/scheduled_download.py >> data/logs/cron.log 2>&1
```

Capítulo 3

Arquitectura del Sistema

3.1 Stack tecnológico

Cuadro 3.1: Dependencias principales

Componente	Versión	Rol
Python	3.11	Runtime principal
FastAPI	$\geq$ 0.110	API REST backend
OpenCV	4.x	Procesamiento de imagen
NumPy/SciPy	–	Álgebra matricial
SAM (Meta AI)	–	Segmentación semántica
PyTorch	$\geq$ 2.0	Backend SAM
Vue 3 + Vite	–	Interfaz web
Tailwind CSS	3.x	Estilos
fpdf2	–	Generación de informes PDF de calibración
PyInstaller + Inno Setup	–	Empaquetado del instalador de escritorio

¿Por qué no hay GDAL ni pyproj en la lista?

El sistema no reproyecta entre sistemas de referencia distintos en tiempo de ejecución: las coordenadas UTM de las varillas de calibración ya vienen dadas en EPSG:25830 (capítulo 4), así que la homografía píxel→UTM que calcula `calculate_homography()` proyecta directamente a ese sistema — `."EPSG:25830"` se escribe como metadato fijo en el GeoJSON de salida (`georef_export.py`), no se calcula con una librería de transformación de CRS. Si en el futuro hiciera falta soportar más de un huso UTM o reproyectar a otro sistema, ahí sí haría falta pyproj.

3.2 Estructura de directorios

Árbol del repositorio

```
cv-lit/
+-- acces_api/              # Módulo Obscape API
|   +-- obscape_api.py      # Cliente REST
|   +-- scheduled_download.py
+-- backend/                # Servidor FastAPI
|   +-- main.py             # App principal + endpoints de cámaras/calibración
|   +-- batch_alignment.py  # Pipeline de alineación masiva
|   +-- auto_mode.py        # Auto Mode: descarga + alineación + análisis
|   +-- config.py           # Constantes globales + CAMERAS
|   +-- desktop_launcher.py # Punto de entrada de la app de escritorio (pywebview)
|   +-- desktop_launcher.spec # Spec de PyInstaller para el .exe principal
|   +-- download_sam.py     # Descargador aparte del checkpoint SAM (opcional)
|   +-- download_sam.spec   # Spec de PyInstaller para download_sam.exe
+-- installer/               # Instalador de escritorio para Windows
|   +-- cv-lit.iss          # Script de Inno Setup
|   +-- output/             # LineaDeCosta-Setup.exe (generado, excl. Git)
+-- calibration/            # Perfiles y máscaras por cámara
|   +-- cam_N_H.npy         # Matriz H 3×3 en float64
|   +-- cam_N_profile.json  # Metadatos, RMSE, nombre de perfil, operador, notas
|   +-- roi_config.json     # ROIs por cámara
|   +-- alignment_masks.json# Zonas SIFT estables
+-- data/                   # Imágenes y resultados (excl. Git) - CVLIT_DATA_DIR
|   +-- camera1/ ... camera6/  # una carpeta por cámara (ver CAMERAS[i]["folder"])
|   |   +-- *.jpg           # capturas originales, nombradas <epoch>_<AAAAMMDD>_<HHMMSS>_<serie>.jpg
|   |   +-- aligned/        # salida del commit de alineación masiva (batch_alignment.py)
|   +-- CAM_1/ ... CAM_6/
|   |   +-- json/           # anotaciones de varillas (una por imagen marcada)
|   +-- system_logs.jsonl   # buffer de actividad que sirve GET /api/logs
+-- docs/                   # Documentación técnica
|   +-- memoria.tex         # Esta memoria  [NUEVO]
|   +-- reports/
+-- frontend/               # Interfaz Vue 3
|   +-- src/
|       +-- App.vue         # Layout raíz + navegación
|       +-- api.js          # API_BASE (VITE_API_URL) usado por todos los componentes
|       +-- components/
|           +-- Dashboard.vue        # Vista general del sistema
|           +-- ImageIngest.vue      # Carga de imágenes
|           +-- Calibration.vue      # Wizard de calibración (7 pasos)
|           +-- BatchAlignment.vue   # Alineación masiva
|           +-- AutoMode.vue         # Auto Mode
|           +-- CoastlineAnalysis.vue# Resultados de análisis
```

```
|           +-- GeoJSONMap.vue       # Mapa GeoJSON exportado
|           +-- Map.vue              # Mapa base reutilizable
|           +-- Cameras.vue          # Gestión de cámaras
+-- homography/
|   +-- align_images.py     # CLI standalone de alineación
+-- proces_images/          # Algoritmos de procesamiento
|   +-- extract_coastline.py
|   +-- segmentation_sam.py
|   +-- georef_export.py
+-- scripts/                # Herramientas de operación
+-- tests/
```

3.3 Pipeline de procesamiento

La figura 3.1 muestra el grafo completo de procesamiento desde la adquisición hasta la exportación del GeoJSON georreferenciado.

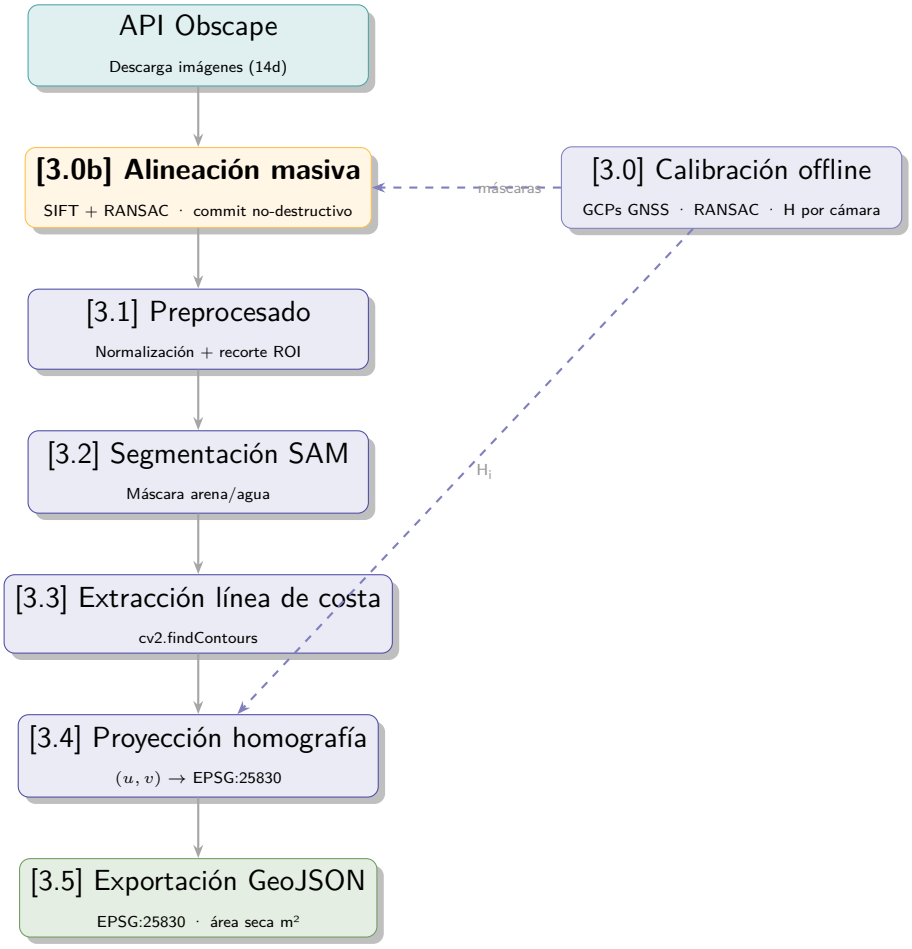


Figura 3.1: Pipeline completo de procesamiento cv-lit

Capítulo 4

Calibración Geométrica

4.1 ¿Qué es la calibración y para qué sirve?

Cada cámara ve la playa en **perspectiva**: un píxel de la imagen no dice, por sí solo, qué punto real de la playa representa. La calibración resuelve exactamente ese problema: encuentra la transformación matemática que convierte un píxel (u, v) de la foto en una coordenada real (X, Y) en metros, en el sistema de referencia EPSG:25830 (ETRS89 / UTM huso 30N).

Esa transformación es una **homografía H**, una matriz 3×3:

$$\begin{pmatrix} X \\ Y \\ 1 \end{pmatrix} \sim \mathbf{H} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \tag{4.1}$$

¿Por qué una homografía y no algo más simple?

Una homografía es la transformación correcta cuando se proyecta un plano (la arena de la playa, que se asume aproximadamente plana) visto desde un punto de vista arbitrario — mezcla en una sola matriz la rotación, traslación, escala *y* la perspectiva de la cámara. Un simple factor de escala metros/píxel no sirve aquí: un píxel cerca de la cámara cubre muchísimo menos terreno real que un píxel cerca del horizonte, y esa relación cambia de forma no lineal con la distancia — es exactamente lo que la homografía sí modela.

Ojo: esta NO es la única homografía del sistema

Este capítulo cubre la homografía de **calibración geométrica**: píxel de la foto → coordenada UTM real. Existe una **segunda homografía**, completamente distinta, en el módulo de alineación masiva (capítulo 5): esa es *imagen→imagen* (corrige el pequeño desplazamiento de la cámara entre dos fotos) y no tiene nada que ver con coordenadas UTM. Las dos se calculan con la misma función de OpenCV (`cv2.findHomography` con RANSAC) pero resuelven problemas distintos, sobre puntos de entrada distintos, y no se deben confundir. Ver la nota comparativa en §4.9.

4.2 De dónde salen los puntos de control (GCPs)

La homografía se ajusta a partir de **GCPs** (*Ground Control Points*, aquí llamados "varillas"): puntos de los que se conocen *las dos* coordenadas del mismo punto físico — dónde cae en la foto (píxel) y dónde está en la realidad (UTM, en metros). Hacen falta un mínimo de 4 pares para que el sistema tenga suficiente información (una homografía tiene 8 grados de libertad); en la práctica se recomiendan 8 o más, y cuantos más se usen, más robusto es el ajuste frente a un punto puntual mal marcado.

Cada GCP llega al sistema por una de estas dos vías:

- **Manual**: clic directo sobre la imagen en la interfaz web (paso *Marcación*, §4.3), con una lupa de precisión (×6) para acertar el píxel exacto en fotos de varios miles de píxeles de ancho.
- **Importado de un levantamiento de campo** (CSV con columna `FILE` que liga cada varilla a la sesión de disparo en la que se topografió): el píxel llega como *aproximado* (varilla "pendiente", marcador ámbar) y hay que confirmarlo con la lupa antes de que cuente para el ajuste — así un píxel mal ubicado en el CSV no puede colar error silenciosamente en la calibración.

¿El cálculo se hace sobre una imagen o sobre todas las de la cámara?

Sobre **una sola imagen**, la que esté seleccionada en ese momento en el paso Marcación —cada imagen tiene su propio conjunto de varillas marcadas y su propia homografía calculada, guardada junto a esa imagen concreta (`data/CAM_X/json/{imagen}.json`, campo `calibration`)—. Lo que SÍ es por cámara es el *resultado activo*: cada vez que se calcula (o recalcula) la homografía de una imagen, esa **H** se copia también al **perfil de la cámara** (`calibration/cam_X_profile.json` y `cam_X_H.npy`), sobrescribiendo la anterior. Es esa copia a nivel de cámara —la última calibración calculada, sea de la imagen que sea— la que usa el análisis de línea de costa (capítulo 10) para *todas* las imágenes nuevas de esa cámara, hasta la siguiente recalibración. En otras palabras: se calibra imagen a imagen, pero solo hace falta una calibración vigente por cámara para poder analizar el resto de sus capturas.

4.3 Flujo actual: calibración interactiva desde la interfaz web

La calibración se hace hoy **enteramente desde el navegador** (pantalla *Calibración*), no con scripts de línea de comandos. El asistente tiene 7 pasos; los tres que corresponden a este capítulo son *Marcación*, *Cálculo* y *Validación*:

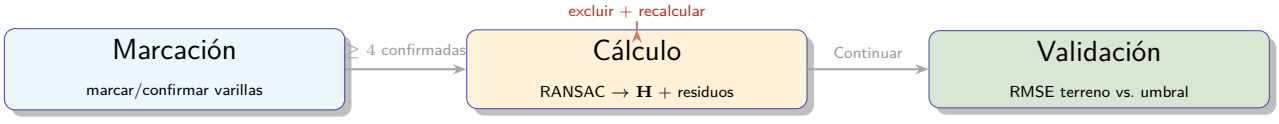


Figura 4.1: Los tres pasos del asistente de calibración implicados en este capítulo

1. **Marcación**: se marca (o confirma) cada varilla sobre la foto. Un clic directo encima

del marcador de una varilla pendiente la confirma en su sitio; un clic en cualquier otro punto de la imagen la reposiciona. Cada varilla confirmada se pinta en **verde**; las pendientes, en ámbar.

2. **Cálculo**: al pulsar *Calcular homografía* se ejecuta el ajuste (§4.4) y aparece una tabla de error de reproyección por varilla — se puede excluir una varilla concreta y recalcular sin ella, de forma iterativa, antes de pasar a Validación. Como la calibración es por imagen (§4.3), este paso incluye también un selector para pinchar entre las distintas fotos de la cámara y comparar el resultado de cada una sin volver a Marcación; la última que se calcule es la que queda activa para la cámara. Cada fila de la tabla muestra, además del error, el **píxel y la coordenada UTM tal y como se marcaron** — para poder verificar cada varilla contra el CSV/Excel de origen si un valor no encaja como se esperaba — y, al pasar el ratón sobre el error, el valor *predicho* por la homografía en ese mismo punto, para ver exactamente qué discrepancia produce el error.
3. **Validación**: resumen final de la calidad de la calibración de esta imagen — RMSE en metros frente al umbral objetivo, y estado *Óptimo*/*Revisar*.

Scripts de calibración retirados

Versiones anteriores de este sistema incluían herramientas de línea de comandos (`calibration_tool.py`, `automate_homography.py`, `recalibrate.py`) para marcar GCPs fuera del navegador. Se han retirado del repositorio: la calibración de producción vive por completo en el backend web (`calculate_homography()`, invocado desde el paso Cálculo) y esas herramientas habían quedado desincronizadas de ese flujo real — `calibration_tool.py` en concreto ni siquiera llegaba a compilar. El flujo de este capítulo es el único vigente.

4.4 Estimación con RANSAC

Estimación de homografía con RANSAC -- calculate_homography() en el backend

```
H, ransac_mask = cv2.findHomography(
    pts_px,              # píxeles marcados/confirmados en la foto
    pts_utm,             # coordenadas UTM del catálogo de varillas
    cv2.RANSAC,
    3.0                  # umbral de reproyección en el dominio UTM: 3 metros
)
```

¿Qué hace RANSAC aquí y qué es un *inlier*?

RANSAC (*Random Sample Consensus*) prueba subconjuntos de varillas y se queda con la homografía que consigue que el mayor número posible de ellas encaje dentro de un margen de error tolerable (aquí, 3 m en UTM). Las varillas que SÍ encajan con esa homografía se llaman **inliers**; las que no —porque su UTM o su píxel están mal, o porque son incompatibles con el resto— quedan como *outliers* y no participan en el ajuste final. Es la misma idea que un "voto mayoritario" geométrico: una sola

varilla mal introducida no puede arruinar la homografía si el resto de varillas son consistentes entre sí. La tabla de residuos del paso Cálculo marca con un ✓verde las varillas que RANSAC aceptó como inlier y con una × roja las que descartó.

4.4.1 Aviso de geometría inestable

Si las varillas marcadas quedan casi todas alineadas a lo largo de una única línea (por ejemplo, una fila de estacas topografiadas siguiendo la orilla, con muy poca variación en la dirección perpendicular), el sistema queda **numéricamente mal condicionado**: existen infinitas homografías que explican igual de bien esos puntos, y una homografía así puede fallar estrepitosamente (RMSE de decenas de metros) aunque el cálculo en sí no produzca ningún error. Se detecta comprobando qué fracción de las varillas activas quedó como inlier de RANSAC:

$$f_{\text{inlier}} = \frac{\text{varillas inlier}}{\text{varillas activas}} \qquad f_{\text{inlier}} < 0{,}6 \;\Rightarrow\; \text{aviso de geometría inestable} \tag{4.2}$$

Cuando esto ocurre, la interfaz muestra explícitamente cuántas varillas de cuántas encajaron y recomienda añadir varillas repartidas en más de una zona/profundidad de la imagen, en vez de dejar solo un RMSE alto sin explicación.

4.5 Dos umbrales que no hay que confundir

La pantalla de calibración maneja **dos umbrales distintos**, en dominios distintos, con propósitos distintos — es la fuente de confusión más habitual al leer los resultados:

Cuadro 4.1: Umbral por varilla (píxeles) frente a umbral de aceptación (metros)

	Umbral	**Para qué sirve**
Error por varilla	5 px (ajustable)	Ayuda visual para *localizar* qué varilla concreta podría estar mal puesta (un misclick, o una varilla del catálogo equivocada). No decide si la calibración es válida.
RMSE terreno	2 m (`RMSE_GOOD_M`)	**Criterio real de aceptación** de la calibración de esta imagen — el mismo valor que pondera el índice de confianza del análisis de línea de costa (capítulo 10).

Por qué el estado ya NO depende de que todas las varillas bajen del umbral en píxeles

En una iteración anterior de este sistema, el indicador “Óptimo / Revisar” exigía que *todas* las varillas individuales tuvieran un error de reproyección por debajo de 1 px. Con fotos de varios miles de píxeles de ancho marcadas a mano, ese criterio era prácticamente imposible de cumplir incluso con una calibración objetivamente

buena: se observó un caso real con RMSE terreno = 0,24 m (muy por debajo del objetivo de 2 m) donde 7 de 8 varillas aparecían igualmente como "sobre umbral". El estado ahora se decide por el RMSE en metros frente a `RMSE_GOOD_M`, que es la métrica que de verdad importa para georreferenciar la línea de costa; el umbral en píxeles sigue existiendo solo como ayuda de diagnóstico por varilla.

4.6 Métricas del ajuste: RMSE y MAE

Para cada varilla activa (no excluida, no pendiente) se calcula el error de reproyección en los dos dominios:

$$e_{\text{px},i} = \left\| \mathbf{H}^{-1}(\text{UTM}_i) - \text{píxel}_i \right\| \tag{4.3}$$

$$e_{\text{m},i} = \left\| \mathbf{H}(\text{píxel}_i) - \text{UTM}_i \right\| \tag{4.4}$$

y a partir de esos residuos, el RMSE (penaliza más los errores grandes — útil para detectar varillas mal marcadas) y el MAE (distancia media, sin elevar al cuadrado):

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2} \qquad \text{MAE} = \frac{1}{N}\sum_{i=1}^{N} e_i \tag{4.5}$$

Tabla histórica — ver estado actual en §12.6

Los resultados de esta sección corresponden a la campaña de calibración de junio 2026 y se conservan como registro histórico. El estado y los RMSE actuales de las 6 cámaras, tras la ronda de refinamiento del 31 de agosto de 2026 (incluida la corrección de cámara 6), están en la tabla de §12.6 (capítulo 12).

4.7 Resultados de calibración (campaña junio 2026)

Cuadro 4.2: Resultados de precisión por cámara — campaña de calibración inicial

Cámara	GCPs	RMSE Cal. (m)	RMSE Val. (m)	RMSE Val. (px)	Estado
CAM 1	51	1.53	**1.45**	19.36	OK
CAM 2	39	2.64	3.45	21.09	Revisar
CAM 3	49	1.61	2.32	38.38	OK
CAM 4	49	2.78	3.06	100.01	Revisar
CAM 5	61	1.45	**1.38**	14.81	OK
CAM 6	69	6.14	4.74	130.62	Critico

Cámaras con error elevado

Los errores de CAM 4 y CAM 6 superan ampliamente el objetivo de 2 m y son consistentes con el problema descrito en §4.4.1: en la sesión de campo correspondiente a CAM 6, las varillas topografiadas quedan casi alineadas en una única línea a lo largo de la orilla (variación de apenas ∼5 m en un eje frente a ∼140 m en el otro), lo que deja la homografía numéricamente mal condicionada. **Acción recomendada:** volver a topografiar varillas para estas cámaras repartidas en más de una zona/profundidad de la imagen (no solo a lo largo de la orilla), y recalibrar desde el paso Marcación de la interfaz web.

4.8 Imágenes de diagnóstico

Las imágenes de la figura 4.2 muestran la reproyección de GCPs sobre las imágenes de referencia, de la campaña de calibración **inicial** (junio de 2026). Los vectores de error se representan magnificados ×10.

Estas cifras son históricas — no las de la calibración actual

Los RMSE de esta figura (1,45–4,74 m) corresponden a la primera campaña de calibración, ya superada por la ronda de refinamiento del 31 de agosto de 2026 (§12.6, tabla 4.2): las 6 cámaras están hoy entre 0,21 y 0,86 m. Las imágenes en sí (`calibration/diagnostics/*.jpg`) no viven en el repositorio de código y, además, los datos de residuo por varilla necesarios para regenerarlas ya no están en este equipo — un *reset* de datos (*Borrar imágenes y resultados*, capítulo 13) borra las varillas marcadas junto con las imágenes, conservando solo el resumen (RMSE, matriz H) en el perfil de calibración. Se dejan aquí como referencia del **procedimiento** de diagnóstico visual, no como estado actual — para regenerarlas hace falta volver a marcar varillas en el paso Marcación y exportar el mapa de cobertura que ya genera la propia aplicación en el botón de informe PDF por cámara del paso Validación (más abajo en este mismo capítulo).

Imagen no disponible en el repositorio: ../calibration/diagnostics/CAM1_diagnostic.jpg

(a) CAM 1 — RMSE val. 1.45 m

Imagen no disponible en el repositorio: ../calibration/diagnostics/CAM2_diagnostic.jpg

(b) CAM 2 — RMSE val. 3.45 m

Imagen no disponible en el repositorio: ../calibration/diagnostics/CAM3_diagnostic.jpg

(c) CAM 3 — RMSE val. 2.32 m

Imagen no disponible en el repositorio: ../calibration/diagnostics/CAM4_diagnostic.jpg

(d) CAM 4 — RMSE val. 3.06 m (revisar)

Imagen no disponible en el repositorio: ../calibration/diagnostics/CAM5_diagnostic.jpg

(e) CAM 5 — RMSE val. 1.38 m

Imagen no disponible en el repositorio: ../calibration/diagnostics/CAM6_diagnostic.jpg

(f) CAM 6 — RMSE val. 4.74 m (crítico)

Figura 4.2: Diagnóstico de calibración — reproyección de GCPs (vectores ×10), campaña de junio de 2026

4.9 Comparación con la homografía de alineación (capítulo 5)

Cuadro 4.3: Las dos homografías del sistema, lado a lado

	Calibración (este capítulo)	**Alineación masiva (cap. 5)**
Dominio	Píxel de la foto → UTM (metros)	Píxel de una foto → píxel de otra foto (misma cámara)
Entrada	Varillas GCP marcadas a mano (píxel + UTM conocidos)	Puntos SIFT emparejados por FLANN entre dos capturas
Umbral RANSAC	3 m (dominio UTM)	5 px (dominio píxel)
Se calcula	Una vez por imagen de referencia, al calibrar	Una vez por cada imagen nueva del lote, contra la base
Para qué sirve	Georreferenciar la línea de costa detectada (obtener m^2, coordenadas reales)	Corregir el pequeño desplazamiento de la cámara (viento, mantenimiento) para que la calibración siga siendo válida en fotos posteriores

Ambas usan `cv2.findHomography(..., cv2.RANSAC, umbral)` porque el problema matemático subyacente es el mismo (encontrar la transformación proyectiva que mejor explica un conjunto de correspondencias de puntos, descartando las atípicas) — pero los puntos de entrada, el dominio del umbral y el propósito final son distintos, y conviene no confundir un "RMSE" o unos "inliers" de una con los de la otra al leer los logs o la interfaz.

Capítulo 5

Módulo de Alineación Masiva

¿Esto se hace sobre una imagen o sobre todas las de la cámara?

Sobre un **lote elegido por el usuario** (típicamente, todas las capturas nuevas de una cámara desde la última alineación), nunca sobre "todas las imágenes" automáticamente sin elegir. El usuario escoge primero una **imagen base** (referencia) y después qué imágenes del lote alinear contra ella; el sistema procesa el lote entero de una sola vez (tarea en segundo plano) y produce un resultado por imagen. La homografía de alineación **no se reutiliza entre lotes**: cada imagen del lote obtiene su propia **H** respecto a la base elegida en ese momento — no hay una única "homografía de alineación de la cámara" persistente, a diferencia de la homografía de calibración del capítulo 4, que sí queda como valor activo por cámara.

5.1 Motivación

El flujo anterior validaba fotogramas de forma **unitaria**: una llamada API por imagen, sin visión global del lote. Esto presentaba tres problemas:

1. **Escalabilidad**: con 24 imágenes/día por 6 cámaras, revisar manualmente cada alineación era inviable.
2. **Detección de deriva**: sin comparativa global era difícil detectar si una cámara había girado ligeramente con el tiempo.
3. **Objetos dinámicos**: SIFT sin restricciones detectaba gaviotas y olas como features, produciendo homografías sesgadas.

5.2 Arquitectura de estados

El módulo implementa una **máquina de estados no-destructiva** de 5 fases:

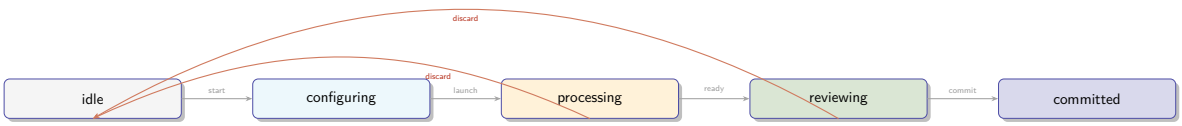


Figura 5.1: Máquina de estados del módulo BatchAlignment

Principio de no-destructividad

Todo el trabajo intermedio vive en el directorio temporal del sistema operativo (`tempfile.gettempdir()/cv_lit_batch/{job_id}/` — p. ej. `/tmp/...` en Linux/macOS, `C:\Users\...\AppData\Local\Temp\...` en Windows). Los originales de cada cámara **nunca** se modifican. La escritura permanente ocurre **únicamente** al llamar al commit explícito.

5.2.1 Estructura del directorio temporal por job

```
{tmp_dir}/cv_lit_batch/{job_id}/
  aligned/    ← imágenes con warpPerspective aplicado (staging)
  diff/       ← mapas de delta por imagen
  blend/      ← blend 50/50 en escala de grises
```

5.3 Pipeline de alineación: algoritmo

5.3.1 Parámetros configurables

Cuadro 5.1: Parámetros del pipeline de alineación (`batch_alignment.py`)

Constante	Valor	Descripción
`SIFT_FEATURES`	3 000	Keypoints máximos por imagen
`LOWE_RATIO`	0.75	Ratio test de Lowe para filtrar matches
`MIN_INLIERS`	15	Inliers mínimos RANSAC (con máscara)
`MIN_INLIERS_FULL`	30	Inliers mínimos sin máscara (fallback)
`RANSAC_THRESH`	5.0 px	Umbral de reproyección RANSAC

De *keypoint* a *inlier*: el camino completo de un punto

Vale la pena distinguir claramente los tres números que aparecen en la interfaz de revisión (§5.5), porque cada uno cuenta algo distinto:

1. **Keypoints** (`SIFT_FEATURES`, hasta 3 000): puntos de la imagen con textura suficientemente distintiva como para reconocerlos en otra foto (esquinas, bordes marcados, estructuras fijas). Se detectan por separado en la imagen base y en cada imagen a alinear.
2. **Matches / coincidencias** (`n_good_matches` en el resultado): de todos los keypoints de ambas imágenes, cuántos el emparejador FLANN considera que *probablemente* son el mismo punto físico, tras aplicar el ratio test de Lowe (0.75) para descartar coincidencias ambiguas. Todavía puede haber matches erróneos aquí (texturas repetidas, reflejos).

3. **Inliers** (`MIN_INLIERS`/`MIN_INLIERS_FULL`): de esos matches, cuántos son geométricamente consistentes con **una única** homografía — es decir, cuántos sobreviven a RANSAC. Es el número que de verdad importa: la homografía final se calcula solo con los inliers. En la interfaz esto se muestra como "*X* / *Y puntos*" (inliers / `n_good_matches`) — un ratio bajo indica que muchos matches iniciales resultaron ser ruido, aunque la alineación en sí haya funcionado bien con los que quedaron.

El δ_{px} (§5.4.3) es un número aparte: no mide cuántos puntos encajaron, sino *cuánto* desplazamiento residual queda de media después de aplicar la homografía — el termómetro de calidad final del resultado, más que del proceso.

5.3.2 Estrategia de dos pasadas

Para combinar la robustez de las máscaras con la resiliencia ante zonas escasas de features:

Lógica de dos pasadas (pseudocódigo)

```
# Pasada 1: con máscara de zonas estables
H, inliers, n_good, fail = _try_align(gray, ref_kp, ref_des,
                                      mask=feat_mask,
                                      threshold=MIN_INLIERS)  # 15
if inliers >= MIN_INLIERS:
    result.used_mask = True
    return H, inliers

# Pasada 2 (fallback): toda la imagen, umbral más alto
H, inliers, n_good, fail = _try_align(gray, ref_kp_full, ref_des_full,
                                      mask=None,
                                      threshold=MIN_INLIERS_FULL)  # 30
result.used_mask = False
```

5.3.3 Flujo completo del pipeline

1. Carga imagen base → extrae keypoints con SIFT (con y sin máscara).
2. Por cada imagen del lote (tarea de fondo asíncrona):
   a) Detecta features → matching FLANN con ratio test de Lowe (0.75).
   b) RANSAC → homografía **H** (dos pasadas).
   c) `cv2.warpPerspective` → imagen alineada → `aligned/`.
   d) `_compute_diff_map` → mapa de delta → `diff/`.
   e) `_blend_grayscale` → blend 50/50 → `blend/`.
   f) `_compute_mean_shift` → métrica escalar δ_{px}.
   g) Actualiza `job.progress_current`.
3. `job.status = "ready"` al finalizar.

5.4 Renderizado del mapa de delta

Sigue siendo código real — ya no tiene botón propio en la interfaz

El backend descrito en esta sección (`_compute_diff_map()`, endpoint `GET /{id}/preview/diff/{fn}`) sigue existiendo tal cual. Lo que cambió es la interfaz: el botón `[Mapa delta]` de `BatchAlignment.vue` (capítulo 7) se retiró a petición del cliente — `viewMode` solo admite hoy `'blend'` y `'aligned'`. La métrica escalar δ_{px} (§5.4.3) sí se sigue mostrando en el filmstrip, sin necesidad de esa vista.

El mapa de delta es una imagen compuesta de tres capas superpuestas que proporciona tanto información visual intuitiva como métricas cuantitativas.

5.4.1 Capa 1 — Heatmap de diferencia absoluta

$$\Delta_{\text{abs}}(i,j) = \frac{1}{3} \sum_{c \in \{R,G,B\}} |I_{\text{ref}}(i,j,c) - I_{\text{ali}}(i,j,c)| \tag{5.1}$$

Se normaliza y se aplica `COLORMAP_HOT` de OpenCV:

Cuadro 5.2: Leyenda del heatmap HOT

Color	Significado
Negro	$\Delta = 0$ — alineación perfecta
Rojo/naranja	Error de intensidad moderado
Amarillo	Error alto
Blanco	Error severo — zona mal alineada

5.4.2 Capa 2 — Divergencia de bordes (overlay cian)

```
edges_ref = cv2.Canny(ref_gray, 40, 120)
edges_ali = cv2.Canny(aligned_gray, 40, 120)
edge_diff = cv2.dilate(cv2.absdiff(edges_ref, edges_ali), kernel_5x5)
composite[edge_diff > 0] = (255, 255, 0)  # cian en BGR
```

Los bordes estructurales que **no coinciden** entre referencia y alineada aparecen en **cian**, identificando exactamente qué objeto se desplazó y en qué dirección.

5.4.3 Capa 3 — Métrica escalar δ_{px} (flujo óptico Farneback)

$$\delta_{\text{px}} = \text{mediana}\left(\sqrt{v_x(i,j)^2 + v_y(i,j)^2}\right) \tag{5.2}$$

donde (v_x, v_y) es el campo de flujo óptico residual post-warp calculado con Farneback.

¿Qué es *delta* y para qué sirve?

Después de aplicar la homografía (`warpPerspective`), la imagen alineada debería quedar prácticamente superpuesta a la referencia. δ_{px} mide cuánto NO ha quedado superpuesta: para cada píxel se estima cuánto se tendría que mover todavía para encajar perfectamente (flujo óptico residual), y se toma la mediana de esos desplazamientos sobre toda la imagen. Es, en la práctica, el indicador de *calidad del resultado* de la alineación — mientras que los inliers (recuadro anterior) miden la calidad del *proceso* que llevó a calcular esa homografía. Un valor bajo (cerca de 0 px) significa que la imagen alineada encaja bien con la referencia; por encima de 3 px empieza a notarse un desajuste visual, y es el umbral que la interfaz usa para pintar el dato en ámbar en vez de verde.

5.5 API REST del módulo

Cuadro 5.3: Endpoints del router `/api/batch/`

Método	Ruta	Descripción
POST	/start	Encola job + lanza pipeline en background
GET	/{id}/status	Polling de progreso (fase processing)
GET	/{id}/results	Lista completa de métricas (fase ready)
GET	/{id}/preview/original/{fn}	Imagen original sin transformar
GET	/{id}/preview/aligned/{fn}	Imagen alineada en staging
GET	/{id}/preview/diff/{fn}	Mapa de delta compuesto
GET	/{id}/preview/blend/{fn}	Blend 50/50 en escala de grises
PATCH	/{id}/approve/{fn}	Override de aprobación individual
POST	/{id}/commit	Two-phase commit → CAM_X/aligned/
DELETE	/{id}	Descarta job y elimina temp dir

5.6 Modelos de dominio

Dataclasses principales (batch_alignment.py)

```python
@dataclass
class AlignmentResult:
    filename:        str
    status:          str     # "ok" | "failed" | "reference"
    inliers:         int = 0 # matches que sobrevivieron a RANSAC
    n_good_matches:  int = 0 # matches FLANN antes de RANSAC (inliers es un
    ↪ subconjunto)
    mean_shift_px:   float = 0.0   # delta - ver §\ref{sec:delta}
    H:               Optional[List] = None  # homografía 3×3 como lista plana
```

```
    approved:        bool = True   # override de aprobación por imagen
    fail_reason:     str = ""      #
    ↪ no_features|low_matches|ransac_failed|bad_lighting:*
    used_mask:       bool = False  # True = pasada con máscara de zonas estables
    lighting_issue:  str = ""      # overexposed|underexposed|low_contrast|""

    def __post_init__(self):
        # Una alineación "failed" cae al frame ORIGINAL sin transformar - nunca
        # debe quedar aprobada por defecto, o el commit la copiaría al
        # directorio final sin que nadie la haya revisado.
        if self.status == "failed":
            self.approved = False

@dataclass
class BatchJob:
    job_id:           str
    cam_id:           int
    base_filename:    str
    image_filenames:  List[str]
    status:           str = "pending"  #
    ↪ pending|processing|ready|committed|failed|discarded
    results:          Dict[str, AlignmentResult] = field(default_factory=dict)
    progress_current: int = 0
    progress_total:   int = 0
```

5.7 Flujo de commit (two-phase write)



Figura 5.2: Flujo de commit no-destructivo

Capítulo 6

Máscaras de Zonas Estables SIFT

6.1 Problema: objetos dinámicos en SIFT

Sin restricciones de zona, SIFT detecta features en **cualquier región** de la imagen, incluyendo objetos dinámicos que producen correspondencias erróneas:

- **Gaviotas y pájaros** en vuelo (zona de cielo/horizonte bajo)
- **Olas y espuma** de mar (bordes dinámicos muy contrastados)
- **Personas y bañistas** en la playa
- **Sombras dinámicas** de nubes
- **Hojas de palmeras** agitadas por el viento (ver corrección CAM 3)

Resultado: matches entre objetos en posiciones distintas entre tomas → homografía sesgada → imágenes "giradas" o desplazadas incorrectamente.

6.2 Solución: máscaras de regiones estables

El fichero `calibration/alignment_masks.json` define zonas con **coordenadas fraccionarias** $(0{,}0 \ldots 1{,}0)$ independientes de la resolución real de la imagen:

$$x_{\text{frac}} = \frac{x_{\text{px}}}{W_{\text{imagen}}}, \quad y_{\text{frac}} = \frac{y_{\text{px}}}{H_{\text{imagen}}} \tag{6.1}$$

`alignment_masks.json - estructura`

```
{
  "CAM_1": {
    "regions": [
      {"label": "horizonte_mar",   "x0": 0.00, "y0": 0.39, "x1": 1.00, "y1": 0.48},
      {"label": "arena_baja",      "x0": 0.00, "y0": 0.73, "x1": 0.40, "y1": 0.88},
      {"label": "vegetacion_dcha", "x0": 0.40, "y0": 0.73, "x1": 1.00, "y1": 1.00}
    ]
  }
```

```
}
```

6.3 Zonas por cámara

Cuadro 6.1: Configuración de máscaras SIFT por cámara

Cámara	Zona 1	Zona 2	Justificación
CAM 1	Horizonte mar (y=0.39–0.48)	Arena baja + Vegetación dcha ampliada (x=0.40–1.00, hasta fondo)	Excluye cielo con gaviotas
CAM 2	Horizonte (y=0.34–0.48)	Vallas+Vegetación izq (x=0.00–0.28, hasta el borde inferior)	Zona estable con vallas y vegetación fija
CAM 3	Horizonte (y=0.45–0.56)	Kiosco esquina inf. dcha (x=0.87–1.00, y=0.80–0.93)	Estructura fija; excluye las palmeras (x=0.72–0.87) que se mueven con el viento
CAM 4	Horizonte (y=0.38–0.52)	Edificio izq (x=0.00–0.28, hasta el borde inferior)	Edificio como anclaje vertical
CAM 5	Horizonte (y=0.36–0.50)	Zona dcha (x=0.68–1.00)	Zona derecha más estable
CAM 6	Horizonte (y=0.40–0.54)	Edificios izq (x=0.00–0.35)	

6.4 Cómo ajustar una máscara

1. Abrir una imagen real de la cámara en un visor que muestre coordenadas de píxel.
2. Identificar las regiones estables (sin gaviotas, olas ni personas).
3. Anotar las coordenadas en píxeles de las esquinas.
4. Convertir a fracciones: $x_f = x_{\text{px}}/W$, $y_f = y_{\text{px}}/H$.
5. Editar `calibration/alignment_masks.json`.
6. Ejecutar el script de verificación:

Script de verificación visual de máscaras

```
import cv2, json, numpy as np

IMG   = "proces_images/data/camera1/IMAGEN_REF.jpg"
MASKS = "calibration/alignment_masks.json"
```

```
CAM   = "CAM_1"

img = cv2.imread(IMG)
h, w = img.shape[:2]
overlay = img.copy()

with open(MASKS) as f:
    cfg = json.load(f)

for r in cfg[CAM]["regions"]:
    x0, y0 = int(r["x0"]*w), int(r["y0"]*h)
    x1, y1 = int(r["x1"]*w), int(r["y1"]*h)
    cv2.rectangle(overlay, (x0,y0), (x1,y1), (0,255,0), -1)
    cv2.putText(overlay, r["label"], (x0+10, y0+30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,0,0), 2)

result = cv2.addWeighted(img, 0.5, overlay, 0.5, 0)
cv2.imwrite("/tmp/mask_check.jpg", result)
print(f"Guardado en /tmp/mask_check.jpg  ({w}x{h})")
```

Capítulo 7

Interfaz Web

7.1 Arquitectura frontend

La interfaz web está construida con **Vue 3** + **Vite** + **Tailwind CSS**, sin gestión de estado global (no Pinia): cada componente mantiene su propio estado reactivo con `ref()` y `reactive()`.

Para iniciar el entorno de desarrollo:

```
# Windows (PowerShell) - workspace de desarrollo vacío en workspace/dev/
.\start_dev.ps1
# Linux/macOS
./scripts/start_dev.sh

# Backend:  http://localhost:8000
# Frontend: http://localhost:5173
```

7.2 Componente BatchAlignment.vue

Componente principal del módulo de alineación masiva. Integrado en el **paso 3** del flujo de calibración (`Calibration.vue`).

7.2.1 Variables de estado

```
const phase    = ref('idle')   // idle|configuring|processing|reviewing|committed
const jobId    = ref(null)     // UUID del job backend activo
const results  = ref([])       // Array de AlignmentResult del lote
const selected = ref(null)     // filename seleccionado en filmstrip
const viewMode = ref('diff')   // 'original' | 'aligned' | 'diff' | 'blend'
```

7.2.2 Props y eventos

Cuadro 7.1: Props del componente BatchAlignment

Prop	**Tipo**	**Descripción**
`camId`	Number	ID de cámara (1–6)
`imageList`	Array	Lista de imágenes disponibles
`initialBase`	String	Imagen base por defecto (del perfil de calibración)
Emit	**Payload**	**Descripción**
`notify`	(msg, type)	Toast del sistema
`committed`	{jobId, committed}	Lote confirmado con éxito
`discard`	—	Usuario descartó el job

7.2.3 Layout de la fase reviewing

Mockup histórico — el modo "Mapa deltaza no está en la interfaz

El botón `[Mapa delta]` de este esquema se retiró de `BatchAlignment.vue` a petición del cliente: `viewMode` solo admite hoy `'blend'` (por defecto) y `'aligned'`. El backend que lo generaba (`_compute_diff_map()`, endpoint `GET /{id}/preview/diff/{fn}`, §5.4.3 más abajo) sigue existiendo y funcionando — solo dejó de tener un botón dedicado en la interfaz web — así que el resto de esta sección y de §5.4.3 describe código real, solo desactualizado en cuántas pestañas hay.

Vista de revisión - esquema

```
+----------------------+-----------------------------------------------------+
| RESUMEN              | [Mapa delta]  [Blend 50/50]  [Alineada]             |
|  Aprobadas: 44       +----------------------+------------------------------+
|  Omitidas:   3       |                      |                              |
|  Fallidas:   0       |     ORIGINAL         |    DELTA / BLEND / ALI       |
+----------------------|     <img>            |    <img>                     |
| FILMSTRIP            |                      |                              |
| • img_001  OK        +----------------------+------------------------------+
| ▷ img_002  ~2.1px    |  Leyenda: ■negro=OK ■rojo=error ■cian=bordes        |
| • img_003  FAIL      |                                                      |
| • img_004  SKIP      |  inliers: 152  delta: 1.4 px  [✓ Aprobada]          |
|   ...                |                                                      |
+----------------------+-----------------------------------------------------+
|  [Confirmar lote (44)]                    [Descartar sin guardar]           |
+-----------------------------------------------------------------------------+
```

7.2.4 Indicadores del filmstrip

Cuadro 7.2: Código de color en el filmstrip

Color	Estado	Condición
Azul	Referencia	Imagen base seleccionada
Verde	OK	$\delta_{\text{px}} \leq 3$
Amarillo	Drift	$\delta_{\text{px}} > 3$
Ámbar	Desaprobada	`approved = false` (manual)
Rojo	Fallida	RANSAC insuficiente

7.2.5 Polling de progreso

El frontend consulta el estado del job cada **800 ms** durante la fase de procesamiento:

```
function _poll() {
  // GET /api/batch/{jobId}/status
  if (data.status === 'ready') {
    clearInterval(pollTimer)
    await _loadResults()   // carga métricas completas
    phase.value = 'reviewing'
  }
  if (data.status === 'failed') {
    clearInterval(pollTimer)
    emit('notify', data.error, 'error')
    phase.value = 'configuring'
  }
  progress.value = data.progress_current / data.progress_total
}
```

7.3 Flujo completo de uso

1. Dashboard → seleccionar cámara → pestaña *Calibración* → paso 3.
2. Elegir imagen base de referencia del lote.
3. Pulsar “Alinear lote completo” → barra de progreso en tiempo real.
4. Revisar filmstrip: verde = OK, rojo = error.
5. Para cada imagen roja o amarilla: inspeccionar mapa de delta / blend.
6. Desaprobar imágenes concretas (opcional) con toggle en el visor.
7. Pulsar “Confirmar lote” → imágenes aprobadas a `data/{carpeta_cámara}/aligned/`.
8. Las imágenes originales en `data/{carpeta_cámara}/` (p. ej. `camera1/`) quedan **intactas** — `aligned/` es una carpeta hermana, no las sustituye.

Capítulo 8

Región de Interés (ROI)

8.1 Criterios de delimitación

Para optimizar la segmentación con SAM y reducir ruido en la extracción de línea de costa, cada cámara tiene una ROI que excluye:

- **Cielo**: píxeles por encima del horizonte.
- **Infraestructura**: paseos marítimos, espigones, edificios fuera de la zona de interés.
- **Mar profundo**: zona sin rotura de onda donde la línea de costa no es visible.

8.2 Coordenadas por cámara

Las ROIs se almacenan en `calibration/roi_config.json` como un diccionario por cámara con claves `x_min`, `y_min`, `x_max`, `y_max` (píxeles, sobre la resolución nativa de captura de cada cámara — **4608×2682**, no 1920×1080). La tabla 8.1 recoge los valores de producción actualmente configurados.

Cuadro 8.1: ROIs de producción por cámara (calibration/roi_config.json)

Cámara	**ROI** $(x_{min}, y_{min}, x_{max}, y_{max})$ **px**	**Justificación**
CAM 1	[0, 993, 4608, 2682]	Excluye horizonte y cielo
CAM 2	[0, 869, 4608, 2682]	Horizonte más bajo
CAM 3	[0, 1118, 4608, 2682]	Ángulo con más cielo visible
CAM 4	[0, 993, 4608, 2682]	Excluye horizonte y cielo
CAM 5	[0, 944, 4608, 2682]	Excluye horizonte y cielo
CAM 6	[0, 1043, 4608, 2682]	Excluye horizonte y cielo

¿De dónde sale un valor por defecto si falta la ROI de una cámara?

Si `roi_config.json` no tiene entrada para una cámara, `analyze_roi()` calcula una por defecto recortando el **40 % superior** de la imagen (`y_min = 0.4 × alto`) — una aproximación razonable al horizonte mientras no se afine a mano, no un

valor definitivo. Los seis valores de la tabla ya están afinados individualmente y sustituyen a ese valor por defecto.

Capítulo 9

Segmentación y Extracción de Línea de Costa

9.1 Segmentación semántica con SAM

Se utiliza **Segment Anything Model (SAM)** de Meta AI en modo *zero-shot*: sin datos de entrenamiento específicos de Guardamar. A diferencia del modo .ªutomático"de SAM (que segmenta todo lo que encuentra en la imagen sin guía), `SAMSegmenter` usa **`SamPredictor` con prompts de punto**: uno o varios puntos semilla por cámara (`CAM_PROMPTS`, coordenadas fraccionarias sobre la ROI) le dicen a SAM qué región debe segmentar como arena seca, y se toma la máscara con mayor `score` de entre las `multimask_output=True` que devuelve.

Segmentación -- extracto real de SAMSegmenter en segmentation_sam.py

```
from segment_anything import sam_model_registry, SamPredictor

sam = sam_model_registry["vit_h"](checkpoint=checkpoint_path)
predictor = SamPredictor(sam)
predictor.set_image(roi_img_rgb)

# input_point/input_label vienen de CAM_PROMPTS (puntos semilla por cámara)
masks, scores, logits = predictor.predict(
    point_coords=input_point,
    point_labels=input_label,
    multimask_output=True,
)
mask = masks[np.argmax(scores)]  # la máscara con mayor score de las candidatas
```

¿Qué pasa si SAM no está disponible o falla?

`analyze_roi()` (backend/main.py) prueba una cascada de 3 niveles antes de rendirse: **Nivel 1** SAM (si el checkpoint está instalado y no lanza excepción) → **Nivel 2** `color_fallback_segmentation()` (heurística HSV, siempre disponible, no depende de PyTorch ni de descargar el checkpoint de ∼2,4 GB) → **Nivel 3** umbral de Otsu sobre el canal V, como último recurso si el color-fallback también devuelve

una máscara vacía. Cada nivel usado queda registrado en el log de actividad (`GET /api/logs`) para poder auditar con qué método se generó cada resultado.

9.2 Extracción de línea de costa

1. Cálculo de contornos sobre la máscara binaria: `cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)`
2. Selección del contorno correspondiente al borde arena húmeda – arena seca.
3. Resultado: polilínea en coordenadas píxel (u, v).

9.3 Proyección a EPSG:25830

Aplicación de la homografía (4.1) para transformar cada punto (u, v) de la polilínea a coordenadas métricas:

pixels_to_utm() - proces_images/georef_export.py

```python
def pixels_to_utm(points_px: list | np.ndarray, H: np.ndarray) -> np.ndarray:
    """points_px: (N,2) pares (col, row). Retorna (N,2) pares (Easting, Northing)
    ↪  en metros."""
    pts = np.asarray(points_px, dtype=np.float64).reshape(-1, 2)
    ones = np.ones((len(pts), 1), dtype=np.float64)
    hom = np.hstack([pts, ones])            # (N, 3)
    proj = (H @ hom.T).T                     # [X, Y, W]
    return proj[:, :2] / proj[:, 2:3]        # normalización homogénea
```

9.4 Exportación GeoJSON

Estructura real del GeoJSON de salida -- export_geojson() en georef_export.py

```json
{
  "type": "FeatureCollection",
  "crs": {
    "type": "name",
    "properties": { "name": "urn:ogc:def:crs:EPSG::25830" }
  },
  "properties": {
    "project": "CV-LIT Guardamar",
    "created": "2026-08-11T17:00:00+00:00",
    "n_features": 1
  },
  "features": [{
    "type": "Feature",
    "geometry": {
      "type": "LineString",
```

```
      "coordinates": [[711234.5, 4209876.3], ["..."]]
    },
    "properties": {
      "ID_Camara":      "CAM_1",
      "Timestamp":      "2026-04-30T12:00:00Z",
      "Confianza_IA":   0.93,
      "Area_Seca_m2":   14250.7,
      "RMSE_m":         0.351,
      "n_puntos":       842,
      "crs":            "EPSG:25830"
    }
  }]
}
```

¿Por qué el CRS aparece dos veces en formatos distintos?

El bloque `crs` de nivel superior (formato URN, el estándar GeoJSON histórico de QGIS/OGC) identifica el sistema de referencia de la colección completa. El campo `çrs": .ᴱᴾSG:25830"` dentro de `properties` de cada feature es una copia en formato corto pensada para lectura rápida (p. ej. en la tabla de `Resultados` del frontend) sin tener que parsear el URN — ambos apuntan al mismo sistema, EPSG:25830.

9.5 Índice de confianza (`Confianza_IA`) y una limitación conocida

Pesos históricos — reequilibrados el 31 de agosto de 2026

Los pesos descritos a continuación (calibración 3, cobertura 1, SAM 2, costa 2) son los que causaban el problema de confianza baja diagnosticado en la limitación de más abajo, y ya no son los que usa el sistema. Los pesos actuales y el porqué del cambio están en §12.6 (capítulo 12). El ejemplo de GeoJSON de §9 anterior tampoco es el que exporta la app real (esa función no está conectada al pipeline en producción) — el esquema real, con el polígono de área seca añadido en esa misma ronda, también está documentado allí.

`confidence_index()` (en `georef_export.py`) combina cuatro señales en una media ponderada — nunca es un solo número aislado:

1. **Calidad de calibración** (peso 3): $\exp(-\text{RMSE}_m/\text{RMSE_GOOD_M})$ — constante para todas las imágenes de una misma cámara hasta la siguiente calibración.
2. **Cobertura de máscara** (peso 1): fracción del frame detectada como arena seca, con la puntuación máxima entre el 15 % y el 40 % de cobertura.
3. **Probabilidad media de SAM** (peso 2): media del canal "seca"del mapa de probabilidad (`prob_map`), calculada sobre la región de interés completa.
4. **Confianza de los puntos de costa** (peso 2, si aplica): media de la confianza por punto de la línea extraída.

Si el resultado queda por debajo de `confidence_min` (ajustado por cámara en `cam_thresholds.py`, típicamente 0.40–0.50), la imagen se rechaza (`reject_reason: "low_confidence:X.XX"`) y no se guarda en el histórico.

Limitación conocida — playas concurridas se rechazan más de lo esperado

Diagnosticado el 2026-08-11 con una ejecución real de Auto Mode (Cámara 1, una semana completa): 62 de 88 imágenes se rechazaron, y en un primer vistazo el patrón horario diario parecía marea alta. Al inspeccionar las imágenes de verdad, la playa estaba seca y visible — el patrón real es **gente y sombrillas en la franja de mayor afluencia**, no marea. Se verificó con dos imágenes reales (misma cámara, mismo día): la cobertura de máscara apenas cambia (0.86 vs 0.90), pero la probabilidad media de SAM (factor 3) cae de 0.20 a 0.11 con playa concurrida — eso explica ~80 % de la diferencia de confianza entre ambas.

Se probaron dos alternativas a la media cruda (mediana, y una media recortada por percentil) y ninguna sirve: la mayoría de la región de interés ya es correctamente "no arena"(mar), así que esas estadísticas colapsan sin señal útil. Una media restringida a píxeles por encima de un umbral absoluto de probabilidad tampoco sirve — invierte el problema y premia a la imagen concurrida (un parche pequeño pero muy seguro de arena libre entre sombrillas puntúa mejor que una playa grande y difusa). Con solo dos imágenes de prueba no hay base suficiente para validar una fórmula nueva que afecta a las seis cámaras en producción, así que se ha dejado documentado en vez de forzar un cambio sin validar. Dos caminos razonables para una futura ronda: (a) recopilar más ejemplos reales de playas concurridas/vacías antes de tocar `confidence_index()`, o (b) atacar la causa raíz enmascarando personas antes de segmentar en vez de ajustar la fórmula de confianza a ciegas.

Capítulo 10

Validación y Métricas de Calidad

10.1 Estrategia de validación

Dos niveles de validación: interna (automática) y de campo (independiente)

Conviene distinguir dos cosas que comparten vocabulario ("RMSE") pero no son lo mismo:

- **Validación interna**, implementada y automática: en el paso Validación de la calibración (capítulo 4), el sistema compara la homografía ajustada contra las *mismas* varillas usadas para calcularla, y expone un RMSE en metros frente a un umbral objetivo (`RMSE_GOOD_M` = 2 m). Es el criterio de aceptación que se usa hoy en producción.
- **Validación de campo**, metodológica (no automatizada en la interfaz): comparar la línea de costa detectada contra puntos GNSS de campo del IEL que **no** hayan participado en la calibración — la referencia más honesta posible, porque no puede estar sesgada por los mismos puntos que ajustaron la homografía. Es el procedimiento que describe el resto de este capítulo, pensado para auditorías periódicas de precisión más que para el día a día del sistema.

Comparación de la línea detectada contra **transectos GNSS independientes** del IEL: puntos **no usados** en la calibración de homografía.

10.2 Métricas

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} d_i^2} \qquad \mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N} d_i \tag{10.1}$$

donde d_i es la distancia mínima en metros del punto GNSS i a la polilínea detectada.

10.3 Criterios de aceptación

Cuadro 10.1: Umbrales de calidad del sistema

Métrica	Objetivo	Condición
RMSE geométrico	< 2 m (`RMSE_GOOD_M`)	Condiciones estándar (12:00h, sin interferencias) — mismo umbral que usa la validación interna del paso Cálculo (§4.5 del capítulo 4)
MAE geométrico	$< 1{,}0$ m	Objetivo secundario
Cobertura temporal	$\geq 90\,\%$	Imágenes a 12:00h procesadas sin error
δ_{px} alineación	< 3 px	Drift residual post-homografía

10.4 Factores de error a monitorizar

- **Calidad de imagen**: niebla, reflejo solar, lluvia, espuma excesiva.
- **Nivel de marea**: afecta la posición real de la línea (calibrar en diferentes estados de marea).
- **Deriva de cámara**: movimiento del soporte por viento, mantenimiento u otras causas.
- **Precisión GCPs**: exactitud del levantamiento GNSS del IEL (instrumental + propagación de error).

10.5 Validación en QGIS

1. Configurar CRS del proyecto en **EPSG:25830**.
2. Cargar GCPs del IEL como capa de puntos.
3. Arrastrar los GeoJSON de `data/processed/lines/` al proyecto.
4. Usar "Distancia a la polilínea más cercana" para calcular d_i.
5. Comparar con ortofoto PNOA (WMS/WMTS del IGN) para validación visual.

Capítulo 11

Módulo de Automatización (Auto Mode)

11.1 ¿Qué problema resuelve?

Sin este módulo, procesar un lote nuevo de capturas de una cámara requiere repetir a mano, imagen a imagen, tres pasos ya vistos en capítulos anteriores: descargar las fotos nuevas de Obscape, alinearlas contra una referencia (capítulo 5) y analizarlas para extraer la línea de costa (capítulo 9). *Auto Mode* automatiza esas tres cosas de punta a punta para un rango de fechas elegido por el usuario, sin reimplementar ninguna: las orquesta, llamando directamente a las mismas funciones que usa el flujo manual.

¿Sobre qué se ejecuta — una imagen / un lote / toda la cámara?

Sobre un **lote acotado por fecha**, elegido explícitamente por el usuario (cámara + rango *desde*/*hasta*) — nunca se descargan ni procesan imágenes fuera de ese rango, y nunca se lanza automáticamente sin que el usuario pulse *Iniciar*. Dentro de ese rango, se procesan **todas** las imágenes nuevas encontradas (las que ya existen en disco se saltan). No sustituye a la calibración (capítulo 4), que sigue siendo un paso manual previo y obligatorio.

11.2 Requisito previo: la cámara debe estar ya calibrada

Auto Mode analiza imágenes con `analyze_roi()`, el mismo endpoint que usa el botón manual *Analizar imagen* — y ese endpoint necesita una homografía activa para la cámara (capítulo 4, §4.2). Por eso, al pulsar *Iniciar* el backend comprueba primero si existe `calibration/cam_{id}_H.npy` o un `H` guardado en el perfil de la cámara; si no, rechaza la petición con un mensaje explícito en vez de descargar imágenes que luego no se podrían georreferenciar:

`_is_calibrated()` -- backend/auto_mode.py

```
1 def _is_calibrated(cam_id: int) -> bool:
2     if os.path.exists(f"{CALIBRATION_DIR}/cam_{cam_id}_H.npy"):
```

```
        return True
    profile = load_json(f"{CALIBRATION_DIR}/cam_{cam_id}_profile.json")
    return bool(profile and profile.get("H"))
```

La calibración inicial de una cámara **no** se automatiza: depende de coordenadas UTM de varillas topografiadas en campo, que no vienen de la API de imágenes de Obscape. Auto Mode empieza donde termina la calibración, no la sustituye.

11.3 Las cuatro fases



Figura 11.1: Pipeline de Auto Mode — cada fase reutiliza código ya existente, sin reimplementar nada

1. **Descarga**: pide a la API de Obscape los puntos de datos de la cámara en [desde, hasta] y descarga las imágenes que aún no existen en disco (mismo criterio de deduplicación que el descargador manual, `acces_api/obscape_api.py`), guardándolas planas en `DATA_DIR/{carpeta_cámara}/` con el nombrado `<epoch>_<AAAAMMDD>_<HHMMSS>_<serie>.jpg` que el resto del backend ya reconoce. Si no hay imágenes nuevas en el rango, el job termina ahí con un mensaje claro (nada que hacer) en vez de seguir con pasos vacíos.
2. **Selección de base**: TODO el lote se alinea contra la `reference_image` **persistente** de la cámara — la misma imagen que fija `/set-reference` y usa por defecto el flujo manual (capítulo 5), *nunca* la imagen más antigua de este lote concreto. Es un cambio deliberado de 2026-08-11: si cada lote semanal se alineara contra su propia base, cada ejecución quedaría en un marco de píxeles distinto al que se usó para calcular la homografía H (que se computa una sola vez, sobre `calibrated_image`) — la H fija aplicada a un marco de píxeles distinto introduce un desplazamiento sistemático que ni el RMSE de calibración (solo se calcula una vez) ni la confianza de SAM detectan. Si la cámara no tiene `reference_image` guardada (calibrada sin pasar nunca por `/set-reference`), cae al comportamiento anterior: la imagen más antigua del propio lote, y un lote de una sola imagen no se alinea (nada dentro del lote contra lo que alinearla).
3. **Alineación**: construye un `BatchJob` igual que el flujo manual de alineación masiva y llama directamente a `batch_alignment._run_pipeline()` y, si el resultado queda `ready`, a `commit_batch()` — sin pantalla de revisión intermedia (§11.4 explica por qué es seguro comitear sin que un humano revise cada imagen).
4. **Análisis**: para cada imagen del lote (la alineada si el commit tuvo éxito, la original si no — `_resolve_camera_image_path()` ya resuelve esa prioridad, capítulo 5), llama a `main.analyze_roi()`: la misma función que ejecuta el botón manual *Analizar imagen*. Segmenta, extrae la línea de costa, la proyecta a UTM con la homografía activa de la cámara y guarda el GeoJSON resultante.

11.4 Por qué el commit automático es seguro sin revisión humana

El módulo de alineación masiva (capítulo 5) se diseñó para uso interactivo: un humano revisa cada imagen alineada y aprueba o descarta antes de comitear. Auto Mode no tiene ese paso — comitea en cuanto el job de alineación termina. Esto es seguro porque `AlignmentResult` fuerza `approved=False` automáticamente en cualquier resultado `"failed"` (ver `__post_init__` en el capítulo 5), y `commit_batch()` solo copia al directorio final las imágenes aprobadas: una alineación fallida queda excluida del commit por sí sola, sin que nadie tenga que marcarla a mano, y el análisis posterior usa entonces la imagen **original sin transformar** para esa captura en concreto (en vez de una alineación mal hecha).

11.5 Modelos de dominio

Dataclasses principales (backend/auto_mode.py)

```python
@dataclass
class ImageResult:
    filename:      str
    captured_at:   Optional[str] = None
    is_base:       bool = False
    align_status:  str = ""    # "" | "ok" | "failed" | "reference"
    confidence:    float = 0.0
    dry_area_m2:   float = 0.0
    rejected:      bool = False
    reject_reason: str = ""
    error:         str = ""

@dataclass
class AutoJob:
    job_id:           str
    cam_id:            int
    from_date:        str
    to_date:          str
    status:           str = "pending"  # pending|downloading|aligning|analyzing|done|error
    step:             str = ""         # descripción legible del paso actual
    progress_current: int = 0
    progress_total:   int = 0
    downloaded:       List[str] = field(default_factory=list)
    results:          List[ImageResult] = field(default_factory=list)
    error_msg:        str = ""
```

11.6 API REST del módulo

Cuadro 11.1: Endpoints del router `/api/automode/`

Método	Ruta	Descripción
`POST`	`/start`	Valida cámara calibrada y encola el pipeline completo en background
`GET`	`/{id}/status`	Polling de progreso (fase actual + contadores)
`GET`	`/{id}/results`	Resultado detallado por imagen, solo cuando `status` es `done` o `error`

Igual que el módulo de alineación masiva, el estado de cada job vive **solo en memoria** (`_jobs`, diccionario global) — no persiste entre reinicios del backend, y el frontend hace polling a `/status` cada 1,5 s mientras el job está en curso.

11.7 Nota de empaquetado: import dinámico y PyInstaller

`_download_new_images()` importa `obscape_api` dentro de la propia función, no como import de módulo al principio del fichero (el mismo patrón que usan `segmentation_sam` y compañía en `main.py`, capítulo 3). El análisis estático de PyInstaller no sigue ese tipo de import, así que sin declararlo explícitamente en `hiddenimports` (`backend/desktop_launcher.spec`) Auto Mode habría funcionado perfectamente en desarrollo y fallado con `ModuleNotFoundError` en el `.exe` empaquetado — un fallo que solo se ve al probar el instalador final, no en desarrollo. Corregido añadiendo `."bscape_api"` a `hiddenimports` y `acces_api/` a `pathex` en el `.spec`.

Capítulo 12

Estado del Proyecto y Roadmap

12.1 Estado actual (agosto 2026)

El proyecto se planteó originalmente con una duración de 6 meses (enero–junio 2026, capítulo 1); en la práctica el desarrollo se extendió hasta agosto de 2026 para completar Auto Mode, el instalador de escritorio y una ronda de pulido orientada a producción que no estaba en el alcance original.

Cuadro 12.1: Estado de hitos del proyecto

Mes	Objetivo	Estado	Nota
1	Preparación datos + diseño interfaces	Hecho	Completado
2	Calibración geométrica	Hecho	6 perfiles generados
3	Segmentación (prototipo)	Hecho	SAM funcional
4	Extracción línea + georreferenciación	Hecho	GeoJSON en EPSG:25830
5	Validación y robustez	Hecho	Las 6 cámaras calibradas y validadas
6	Integración y entrega	Hecho	Instalador de escritorio generado y probado

12.2 Completado en la iteración de junio 2026

- Módulo de alineación masiva no-destructiva (`batch_alignment.py`)
  - Router FastAPI `/api/batch/` con 10 endpoints
  - Pipeline async SIFT+FLANN+RANSAC sobre lote completo
  - Mapa de delta compuesto (heatmap HOT + edge-diff cian + métrica Farneback)
  - Two-phase commit: staging `/tmp/` → escritura permanente sólo en commit
- Interfaz de validación por lotes (`BatchAlignment.vue`)
- Corrección de máscaras SIFT para CAM 1 (problema de gaviotas)
- Integración en paso 3 del flujo de calibración (`Calibration.vue`)

- Documentación completa (esta memoria)

12.3 Fase 3 — Automatización y robustez (completada)

1. **Procesamiento por lotes**: resuelto por **Auto Mode** (`backend/auto_mode.py`, capítulo 11) — descarga, alinea y analiza un rango de fechas completo de una cámara de punta a punta, bajo demanda.
2. **Filtros de calidad**: `analyze_roi()` rechaza automáticamente imágenes con confianza de SAM por debajo del umbral configurado (`conf_min`), registrando el motivo en el log de actividad.
3. **Persistencia GeoJSON**: `_append_history()` acumula cada análisis en `coastline_history_cam` consumido por el gráfico "Tendencia Histórica"del Dashboard (`GET /api/historical-data`).
4. **Recalibración de las 6 cámaras**: completada — todas por debajo del objetivo de RMSE < 2 m (`RMSE_GOOD_M`, capítulo 4).

12.4 Fase 4 — Entrega e integración (completada)

1. Dashboard de histórico: gráfica de evolución del área seca ya en producción (`Dashboard.vue` + `GET /api/historical-data`).
2. Exportación de informes: `GET /api/report` (CSV/JSON) desde el Dashboard, más un informe PDF individual por cámara con el resumen de su calibración (`GET /api/cameras/{id}/calibration-report.pdf`, capítulo 4).
3. Manual de usuario: `docs/user_manual.md`, con la guía paso a paso del asistente de calibración de 7 pasos y de Auto Mode.
4. Instalador de escritorio para Windows: `installer/cv-lit.iss` + `backend/desktop_launcher.sp` compilado y probado (`installer/output/LineaDeCosta-Setup.exe`) — ver capítulo 14.

12.5 Ronda de pulido para producción (julio–agosto 2026)

Tras cerrar las Fases 3 y 4, una auditoría completa del proyecto orientada a llevarlo a producción añadió, sin estar en el roadmap original:

- Endurecimiento de seguridad: `CORS allow_credentials=False`, validación de `PUT` en varios endpoints, sanitización de nombres de fichero (`_safe_filename()`), límite mínimo de subida (`MIN_UPLOAD`), y eliminación de credenciales de Obscape hardcodeadas (ahora por variables de entorno, `acces_api/obscape_api.py`).
- Eliminación de código y scripts no usados (rutas de alineación duplicadas, herramientas de calibración de un solo uso, la vista rectificada UTM"nunca terminada).
- Aviso de geometría inestable en el cálculo de homografía (`geometry_warning`, capí-

tulo 4) y su indicador correspondiente en el stepper del asistente.

- Suite de tests ampliada: `tests/test_auto_mode.py` y `tests/test_geometry_warning.py`, cubriendo Auto Mode y el aviso de geometría sin depender de la red real de Obscape.
- Esta memoria: reescrita y ampliada de forma sustancial (capítulos 4, 5, 11 y auditoría de coherencia del resto de capítulos).

12.6 Ronda de refinamiento — 31 de agosto de 2026

Sesión centrada en diagnosticar por qué el `.exe` instalado seguía dando confianza baja pese a los ajustes de la ronda anterior, y en una auditoría línea por línea de todo el código (backend, frontend y scripts sueltos) buscando incoherencias. Los números de RMSE de la tabla 4.2 y los pesos de `confidence_index()` descritos en §9.5 corresponden a la campaña de junio 2026 y a la ronda de julio–agosto respectivamente: **ambos quedan obsoletos a partir de aquí** y se conservan solo como registro histórico.

12.6.1 La confianza baja tenía tres causas independientes, no una

Bug de empaquetado: el checkpoint de SAM nunca se encontraba en el `.exe`

`download_sam.py` guarda el checkpoint de SAM junto al propio `.exe` (`os.path.dirname(sys.executable)`), pero `main.py` lo buscaba en `BASE_DIR` (`sys._MEIPASS`) — en PyInstaller *onedir* a partir de la versión 6, `sys._MEIPASS` apunta a la subcarpeta `_internal/`, **no** a la carpeta donde vive el ejecutable. El resultado: `SAMSegmenter.__init__` no lanzaba ninguna excepción al no encontrar el fichero (se queda con `predictor=None` en silencio), así que el log decía engañosamente "SAM Segmenter cargado con éxito" mientras la app caía sin avisar al respaldo de color/Otsu en *todas* las imágenes del `.exe` instalado. Corregido con una constante `EXE_DIR` nueva en `main.py`, distinta de `BASE_DIR`, y con un log honesto que sí distingue .[en]contrado"de "no encontrado". Verificado extrayendo el bytecode compilado del `.exe` real (no solo el código fuente) y relanzando la app instalada: la confianza de cámara 1 pasó de 0,30 (rechazada) a 0,66.

El mismo patrón —una ruta calculada con `sys.executable` en un sitio y con `sys._MEIPASS` en otro— apareció una segunda vez en `acces_api/obscape_api.py` (`LOG_DIR`, usado por *cada* `ObscapeClient()`, incluido el botón "Probar conexión"de Configuración y cada ejecución de Modo automático) y ya se había corregido antes, en la ronda anterior, para `roi_config.json` (capítulo 8) — sin ese fichero, SAM colocaba sus puntos de referencia sobre la imagen sin recortar. Las tres causas eran reales y ortogonales entre sí; arreglar solo una dejaba la confianza baja por las otras dos.

12.6.2 Un cuarto problema, específico de cámara 6: un punto de calibración con `utm=[0,0]`

Al verificar cámara 6 en el `.exe` real se encontró un RMSE de **1 425 242,98 metros** — una homografía degenerada. La causa: entre las varillas confirmadas de la imagen

de referencia activa había una, `VARILLA_11`, con coordenada UTM `[0, 0]` (un valor de prueba que nunca se limpió, del mismo tipo que el ya documentado para cámara 2 en rondas anteriores). Se eliminó mediante la API real de anotaciones (no editando el JSON a mano) y el RMSE de cámara 6 bajó a 0,25 m — mejor que el resto de cámaras.

12.6.3 Reequilibrio de `confidence_index()` (continuación de §9.5)

Diagnosticado con las 6 cámaras ya recalibradas con datos reales: el peso 3 de la calidad de calibración (caída exponencial dura) penalizaba dos veces cualquier RMSE .ªceptable pero no excelente"que ya había pasado el umbral de aceptación por separado, y la probabilidad media de SAM (peso 2) se diluía con el agua/cielo de la ROI aunque la playa se detectara perfectamente. Pesos nuevos: calibración 1, cobertura de máscara 2 (la señal más fiable de las tres en los casos reales comprobados), probabilidad SAM 1, confianza de costa 2.

12.6.4 Otros arreglos reales de esta ronda

- **Prompt de SAM**: de 1 punto de arena a 3, repartidos en horizontal a la misma altura — con un solo punto, una persona o sombrilla justo encima hacía perder la referencia de .^estoto es arena.^en toda la imagen. Se probó también añadir una caja delimitadora junto a los puntos: **empeoraba** el resultado (la arena seca es una franja diagonal alargada, no un objeto compacto tipo caja) y se descartó, verificado con overlay visual antes/después.
- **Polígono de área seca en el GeoJSON**: cada análisis exporta ahora, además de la línea de costa, un polígono cerrado con el contorno completo de la máscara de arena detectada — mismo dato que ya se usaba para calcular los m^2, ahora también expuesto como geometría (relleno translúcido en el mapa, para distinguirlo de la línea).
- **Dashboard**: Ïmágenes procesadasz "Área seca media.^estaban literalmente hardcodeados en el backend (506 y "23 480", fijos, nunca reflejaban los datos reales) — ahora se calculan de verdad sobre el histórico.
- **Borrado de una imagen** (`DELETE /api/cameras/{id}/images/{filename}`) dejaba huérfanas su copia alineada y su anotación de varillas; ahora limpia las tres cosas.
- **"Procesar selección.^en Carga de imágenes** solo analizaba la primera imagen del grupo marcado, aunque el botón mostraba el total seleccionado — ahora procesa todas.
- **`.gitignore`** excluía `CAM_N/json/` bajo una regla pensada para cuando esa carpeta solo guardaba metadatos descartables de la API — esa carpeta es hoy donde se guardan las varillas GCP confirmadas y la homografía calculada de cada imagen. Con la regla vieja, git nunca veía ese trabajo: 38 ficheros de calibración real vivían solo en disco, sin red de seguridad de control de versiones.
- `backend/requirements.txt` no listaba `pywebview` ni `pyinstaller` (instalados en el entorno de desarrollo pero ausentes del fichero) — un `pip install` limpio no habría podido ni arrancar ni construir la app de escritorio.

- Depuración de imágenes sin respaldo en `GCP.csv`: cámara 1 conservó 6 de 95 (las 89 restantes eran capturas reales de monitorización horaria, movidas a una carpeta de respaldo, no borradas) y cámara 5 conservó 10 de 11.

12.6.5 Estado final de calibración de las 6 cámaras (verificado en el `.exe` real)

Cuadro 12.2: RMSE tras la ronda de refinamiento del 31 de agosto de 2026

Cámara	RMSE (m)	Nota
CAM 1	0,24	OK
CAM 2	1,37	OK en RMSE, pero se sigue rechazando por `área_no_fiable` — necesita más varillas repartidas (dato de campo, no bug)
CAM 3	0,86	OK
CAM 4	0,34	OK
CAM 5	0,72	OK
CAM 6	0,25	OK — antes crítica, corregida (§anterior)

Las 6 cámaras quedan por debajo del objetivo `RMSE_GOOD_M` = 2 m. Cámara 2 es el único caso pendiente real: su homografía es correcta pero el área detectada supera el umbral de plausibilidad porque las varillas topografiadas no cubren suficiente extensión de la playa — requiere una nueva sesión de campo, no un cambio de código.

12.7 Ronda de funcionalidades nuevas — reunión de equipo, 11 de septiembre de 2026

Reunión de demostración con Alejandro Triviño, Andrés Fuster Guilló y Jorge Azorín López: se mostró la versión desarrollada de la aplicación y se recogieron los puntos de acción para la siguiente iteración (guía de uso completa en el capítulo 13).

- **Métricas basadas en transectos** (§13.4.1): el área en m^2 se consideró poco fiable como indicador ("olvidaros de la cuestión de los cálculos de áreas") — se sustituye en prioridad por la anchura de playa medida desde un punto de referencia fijo hasta la línea de costa detectada.
- **Sección temporal de entrenamiento** (§13.4.2): antes de poder automatizar dónde está la "línea húmeda" real (objetivo final de la segmentación, §13.4.3), el equipo de campo necesita marcar a mano, sobre ∼20 imágenes por cámara, dónde debería estar esa línea.
- **Edición manual de la segmentación automática** (§13.2.3): poder corregir a mano la línea que da SAM cuando se equivoca, conservando la corrección como dato de entrenamiento — unificado con el punto anterior en el mismo almacén.

- **Revisar/descartar antes de procesar**: tanto en *Carga de imágenes* (visor de previsualización a tamaño completo) como en *Entrenamiento* (descarte blando y reversible, igual que descartar una varilla de calibración).

Las cuatro piezas quedaron implementadas, compiladas en el `.exe` y verificadas contra la app real (incluyendo un análisis SAM real end-to-end para transectos y edición manual). Queda pendiente, a propósito, el punto que la propia reunión dejó abierto: cambiar el objetivo real de segmentación de "arena seca" a una línea de costa real — depende de los datos que salgan de la sección de Entrenamiento, no es un cambio de código que se pueda hacer a ciegas. Esa referencia exacta se afinó más adelante (§13.4.3): no es la transición húmedo/seco sino la línea que separa el mar de la arena.

Capítulo 13

Guía de Operación

Este capítulo es la guía **definitiva de uso** de la aplicación instalada — pensada para quien va a **usar** la app, no para quien la desarrolla. El flujo de trabajo para quien sí toca código (entorno de desarrollo, herramientas CLI, tests) está aparte, en §14.6.

13.1 Abrir la aplicación

Para el uso normal no hace falta Python, Node.js ni ninguna terminal: se instala el ejecutable de Windows (capítulo 14, §14.5) y se abre como cualquier otro programa.

1. Ejecutar `LineaDeCosta-Setup.exe` y seguir el asistente de instalación.
2. Abrir **"Línea de Costa"** desde el menú Inicio o el acceso directo del escritorio.
3. La primera vez puede tardar unos segundos en aparecer la ventana: el propio ejecutable arranca primero el backend en segundo plano y luego abre la ventana nativa (*pywebview*) apuntando a él — no hace falta ningún navegador aparte.

No usar el acceso directo "Descargar modelo de IA (SAM)"para el uso normal

El instalador crea, además del acceso a la app, uno secundario en el menú Inicio para *forzar* la descarga del checkpoint de SAM (∼2,4 GB) sin abrir la ventana — solo hace falta si SAM no se instaló bien la primera vez o se quiere volver a descargar aposta. Abrirlo por error solo muestra una consola que imprime una línea y se cierra sola; no es un fallo, pero confunde si se pulsa pensando que es la app.

Transectos y Entrenamiento: ocultos del menú a día de hoy (23/09/2026)

A petición del usuario, las pantallas *Transectos* (§13.4.1) y *Entrenamiento (temp.)* (§13.4.2) **no tienen botón de acceso en el menú lateral** en esta versión — se ocultaron, no se eliminaron: el componente, los endpoints de backend y los datos ya guardados siguen intactos y funcionando (un transecto ya marcado, por ejemplo, se sigue calculando en cada análisis). Esta guía las documenta igualmente porque pueden reactivarse en cualquier momento — basta con descomentar el botón correspondiente en `App.vue` (marcado con un comentario que explica por qué está apagado) y volver a compilar. Las capturas de este capítulo corresponden a la interfaz **tal y como se ve ahora mismo**, sin esos dos botones.

13.2 Recorrido por pantallas

El menú lateral agrupa las pantallas en dos bloques: **Proceso** (el trabajo diario) y **Sistema** (configuración y mantenimiento, se toca con menos frecuencia).

13.2.1 Dashboard

Vista general: estado de calibración de las 6 cámaras, gráfica de evolución histórica del área seca y accesos rápidos a cada cámara.

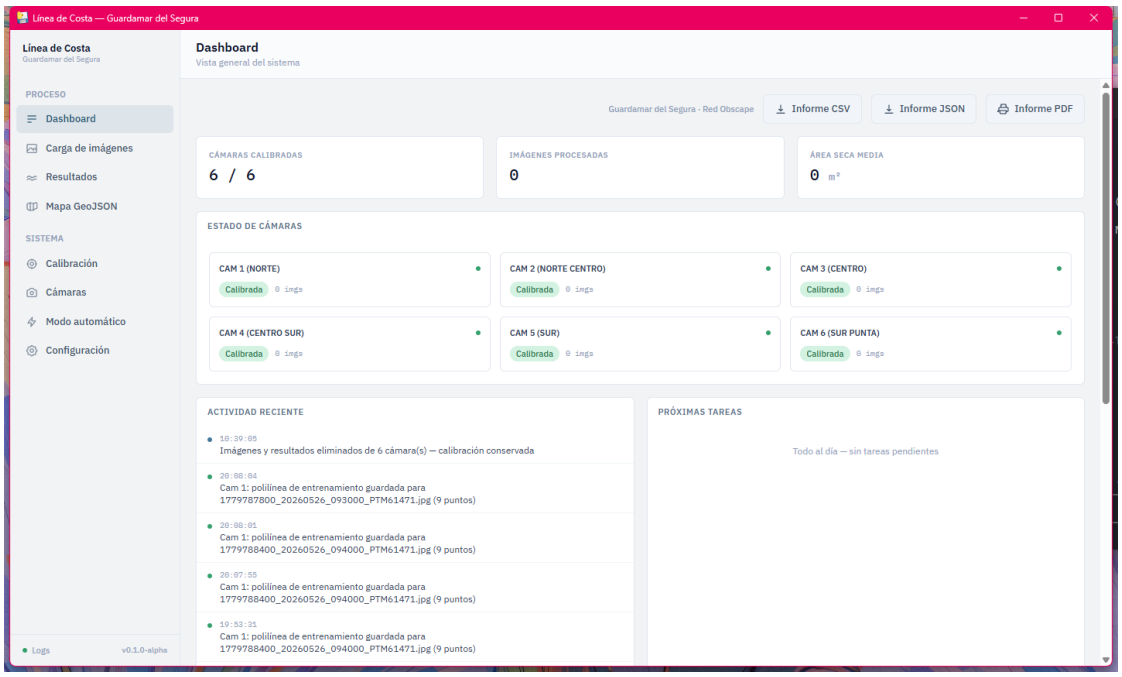


Figura 13.1: Dashboard, con las 6 cámaras calibradas y el botón *Informe PDF* sustituyendo al antiguo *Imprimir Reporte*.

13.2.2 Carga de imágenes

Pantalla para gestionar qué imágenes tiene cada cámara y lanzar su análisis.

- **Subir imágenes**: arrastrar archivos a la zona punteada, o hacer clic para elegir con el explorador.
- **Filtro de fechas**: acota la cuadrícula de miniaturas a un rango *desde/hasta*.
- **Selección**: clic sobre una miniatura para marcarla/desmarcarla; *Todas* / *Ninguna* / *Solo H:00h* (la hora con más fotos del filtro actual) la rellenan de golpe.
- **Previsualizar antes de procesar** (nuevo, 2026-09-14): el icono de lupa que aparece al pasar el ratón sobre una miniatura abre la foto real a tamaño completo — la miniatura es demasiado pequeña para juzgar si una imagen sirve (niebla, cámara movida, algo tapando la zona). Desde ese mismo visor se puede **Descartar** (elimina el archivo) o **Cerrar** sin tocar nada.
- **Punto verde** en una miniatura: esa imagen ya tiene un resultado aceptado en el histórico.

- **Botón de la papelera** (esquina inferior derecha de la miniatura, aparece al pasar el ratón): borra la imagen directamente, con confirmación.
- **Botones de proceso**, de izquierda a derecha:
  - *Reanalizar todas (N)* — repite el análisis de TODAS las imágenes del filtro actual, incluidas las ya analizadas. Hace falta tras una mejora de la segmentación, para refrescar resultados antiguos.
  - *Analizar pendientes (N)* — solo las que nunca se han analizado.
  - *Procesar selección (N)* — solo las marcadas a mano.

  Las tres procesan las imágenes **una a una** (SAM no admite lote) y al terminar llevan a *Resultados* con la última imagen procesada.

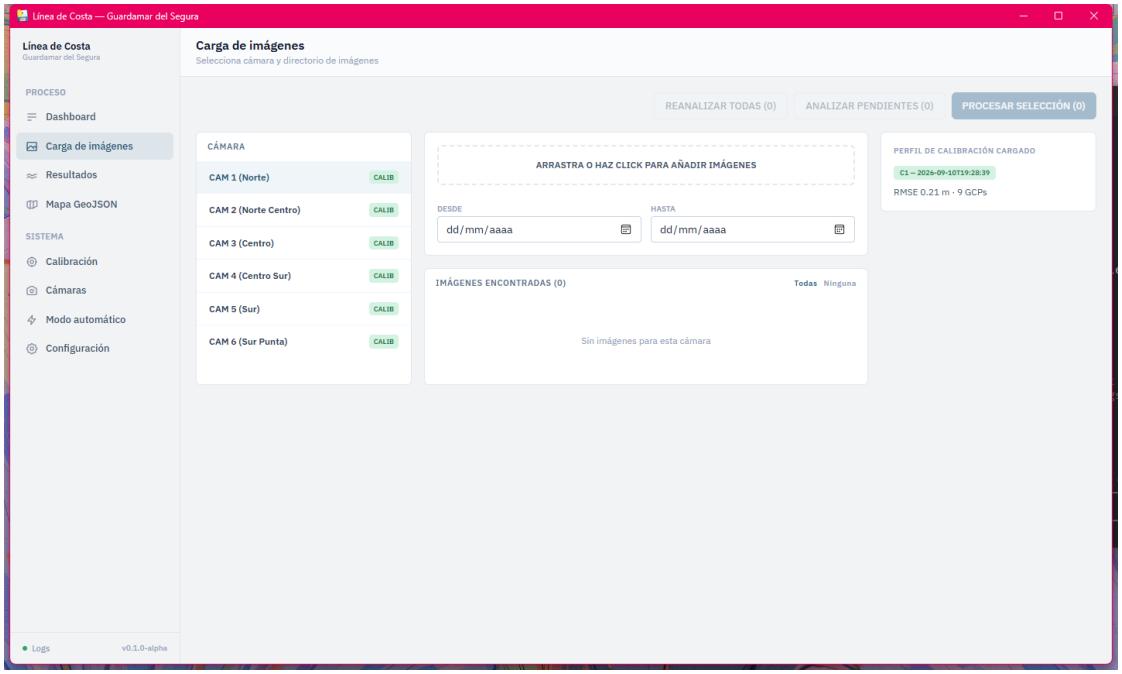


Figura 13.2: Carga de imágenes — selector de cámara, zona de subida, filtro de fechas y perfil de calibración cargado.

13.2.3 Resultados

Pantalla principal de análisis: seleccionar cámara e imagen, pulsar *Analizar imagen* y revisar la línea de costa detectada y sus métricas.

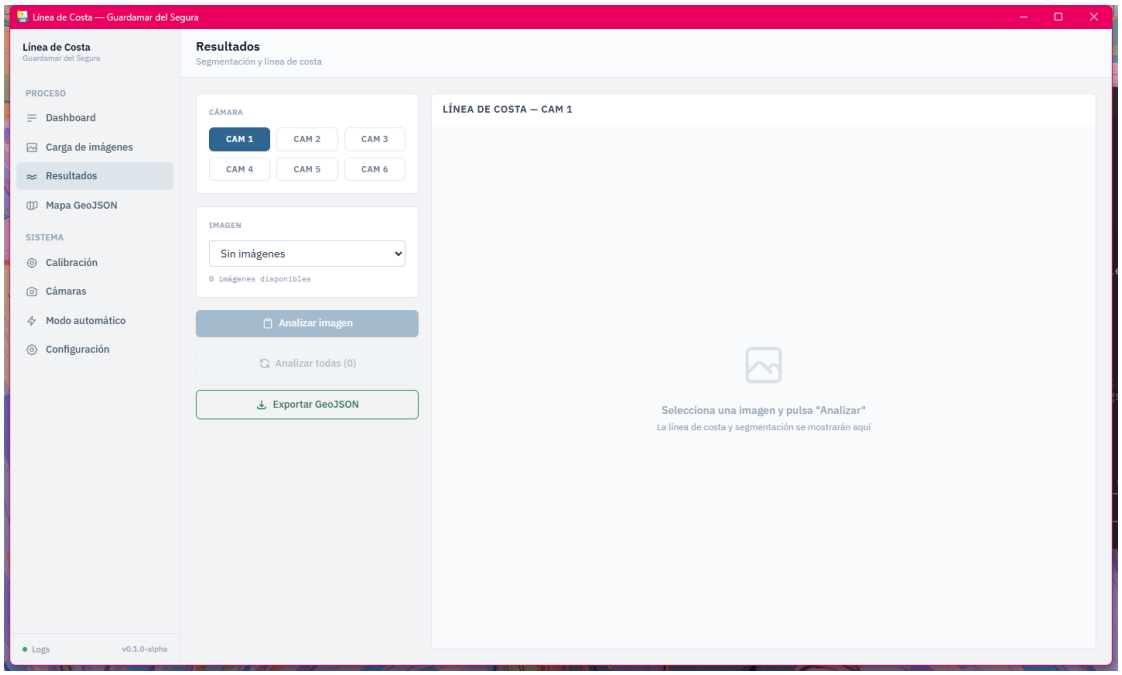


Figura 13.3: Resultados — selector de cámara/imagen, panel de métricas y visor de la línea de costa.

- **Flechas de navegación** (junto al título): pasan a la imagen anterior/siguiente de la lista. Si esa imagen **ya tiene** un resultado aceptado en el histórico, se muestra al instante, sin volver a ejecutar SAM (§13.2.3); si nunca se analizó, se lanza un análisis real, igual que con el botón de siempre.
- **Analizar todas (N)**: analiza de golpe todas las imágenes de la cámara seleccionada, sin salir de esta pantalla — para ver la evolución completa sin ir a Carga de imágenes.
- **Panel de métricas**, de arriba a abajo:
  1. **Confianza** — barra verde/ámbar/rojo.
  2. **Anchura de playa (transectos)** — un valor en metros por cada transecto marcado para esta cámara (§13.4.1). Con la pantalla de Transectos oculta del menú a día de hoy, este apartado no aparece salvo que ya hubiera transectos marcados de antes; sigue siendo, conceptualmente, el indicador pensado para reemplazar al área en fiabilidad.
  3. **Área seca (m²)** — se mantiene visible como referencia, no se ha eliminado nada.
  4. **Puntos UTM** y **Timestamp**.
- **Editar línea manualmente** (nuevo, 2026-09-14, botón junto al título, solo visible con un resultado aceptado): corrige a mano la línea que dio SAM cuando se equivoca — ver §13.2.3.
- **Exportar GeoJSON**: descarga el histórico completo de la cámara en un `.geojson` (EPSG:25830), listo para abrir en QGIS.

Caché por imagen: por qué las flechas no vuelven a lanzar SAM

Cada análisis se guarda con su nombre de fichero, tanto la imagen con la línea dibujada (`GET /api/cameras/{id}/analysis-result?file=...`) como el resultado numérico (`GET /api/cameras/{id}/images/{filename}/cached-result`, que lee directamente el histórico). Navegar con las flechas a una imagen ya analizada consulta ese resultado guardado en vez de relanzar SAM — instantáneo, y no satura el log con análisis repetidos de fotos que no han cambiado.

Edición manual de la línea detectada

Pedido en la reunión del 11/09/2026

"Permite editar manualmente las segmentaciones automáticas y conserva las correcciones como datos de entrenamiento."

Al pulsar *Editar línea manualmente*, la imagen mostrada cambia de la visualización (con la línea ya "quemada" en el JPG) a la **foto original**, con la línea detectada por SAM superpuesta como puntos de control editables —`analyze_roi()` simplifica la línea píxel a píxel (que sale dentada) con `cv2.approxPolyDP` a un puñado de puntos manejables, no cientos.

- **Clic** sobre la imagen: añade un punto nuevo.
- **Arrastrar** un punto existente: lo mueve.
- **Doble clic** sobre un punto: lo borra.
- *Guardar corrección*: envía los puntos corregidos al **mismo almacén** que usa la sección de Entrenamiento (§13.4.2) — una corrección hecha aquí y una polilínea marcada a mano en Entrenamiento acaban en el mismo conjunto de datos, no en dos sitios separados.
- *Cancelar*: descarta los cambios sin guardar nada.

13.2.4 Mapa GeoJSON

Visualiza sobre un mapa real las líneas de costa detectadas de todas las cámaras, en EPSG:25830.

13.2.5 Calibración

Asistente de 7 pasos para establecer la homografía píxel → UTM de cada cámara — detallado en el capítulo 4. No hace falta repetirlo salvo que la cámara se desplace de forma notable (viento fuerte, limpieza brusca) o se añada una cámara nueva.

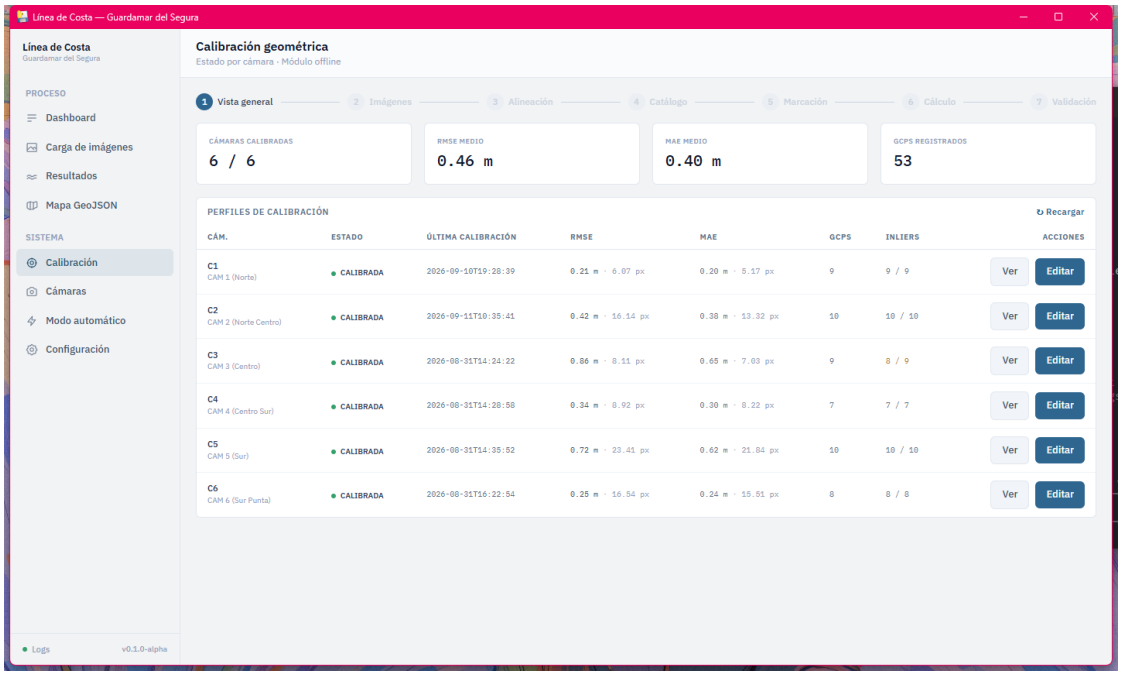


Figura 13.4: Calibración — vista general con el estado real de las 6 cámaras: RMSE, MAE, GCPs e inliers.

13.2.6 Cámaras

Configuración de hardware por cámara: lente, ROI (región de interés) y ubicación.

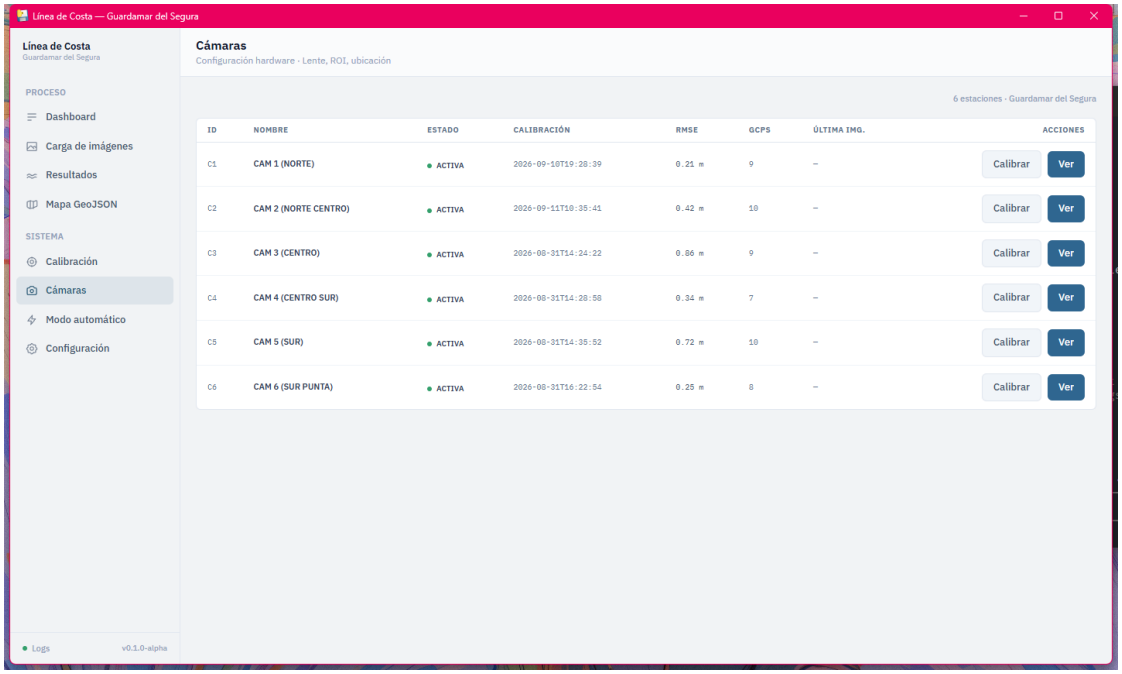


Figura 13.5: Cámaras — estado de las 6 estaciones (RMSE, GCPs, fecha de calibración).

13.2.7 Modo automático

Automatiza de punta a punta, para un rango de fechas, lo que *Carga de imágenes* y *Resultados* hacen a mano: descarga de la API de Obscape, alineación y análisis — capítulo 11. Requiere una cámara ya calibrada.

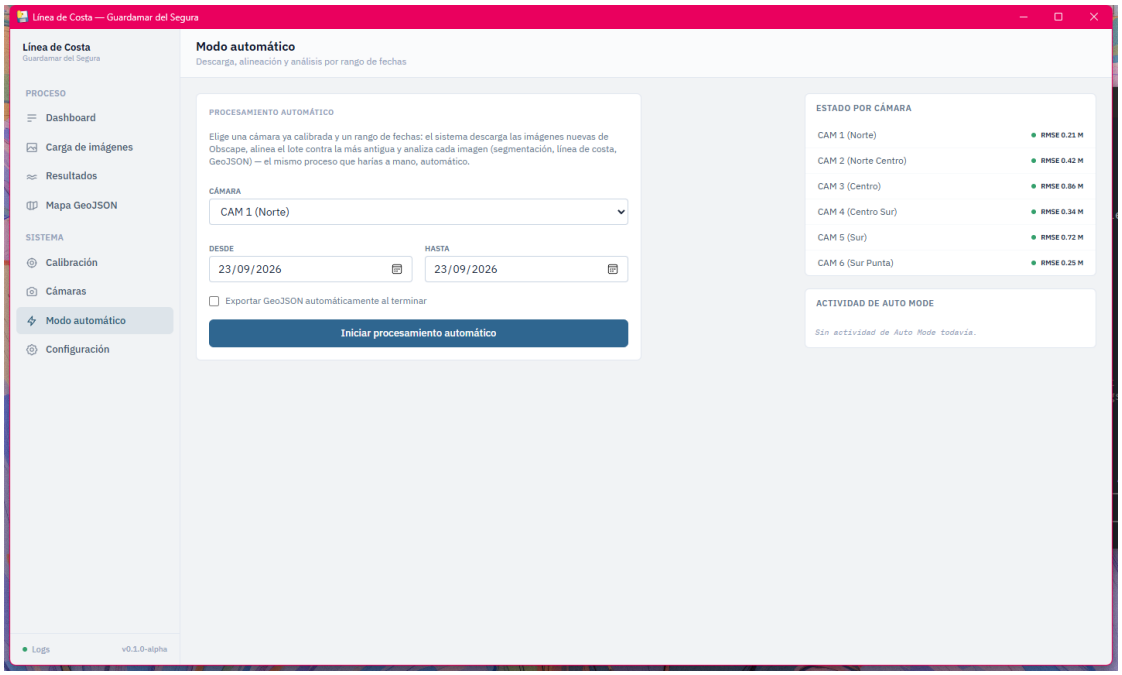


Figura 13.6: Modo automático — panel de procesamiento por rango de fechas y estado de RMSE por cámara.

13.2.8 Configuración

- **Credenciales de Obscape**: usuario y clave de API para descargar imágenes — se guardan solo en este ordenador.
- **Ubicación de los datos**: rutas reales en disco de imágenes y calibración de este ordenador, con botón *Copiar*.
- **Borrar imágenes y resultados**: acción destructiva e irreversible (requiere escribir `BORRAR` para confirmar) — elimina imágenes, varillas marcadas e histórico de las 6 cámaras. **La calibración no se toca.**

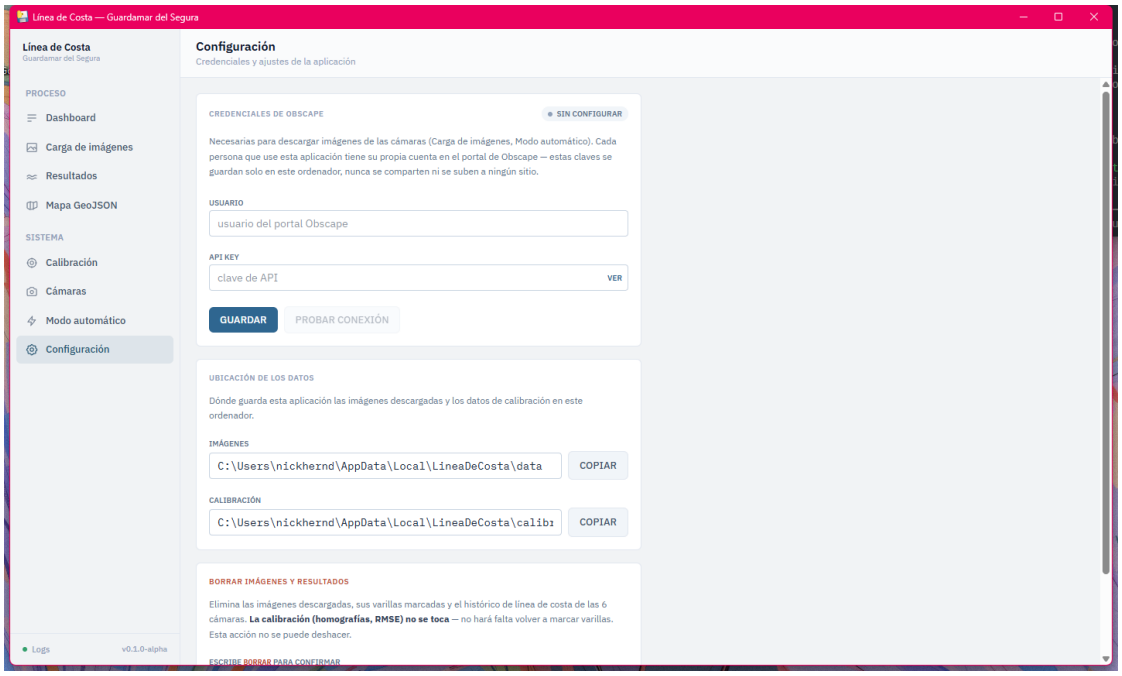


Figura 13.7: Configuración — credenciales de Obscape, ubicación de los datos y borrado de imágenes/resultados. La tarjeta de Entrenamiento que mostraba la ruta de esa sección (§13.4) está oculta con el mismo criterio.

13.3 Flujo de trabajo recomendado (día a día)

1. *Modo automático* (o *Carga de imágenes* + *Analizar pendientes*) para procesar las imágenes nuevas de cada cámara ya calibrada.
2. Revisar en *Resultados* los rechazos y las confianzas bajas; si la línea detectada se equivoca de forma puntual, corregirla con *Editar línea manualmente* en vez de descartar el análisis entero.
3. Exportar el GeoJSON o el informe PDF cuando haga falta compartir resultados.

13.4 Funcionalidades en pausa

Sin botón de acceso en el menú a día de hoy (23/09/2026)

Las dos pantallas de esta sección — *Transectos* y *Entrenamiento (temp.)* — no tienen botón de acceso en el menú lateral en esta versión: se ocultaron a petición del usuario, **no se eliminaron**. El componente, los endpoints de backend y los datos ya guardados siguen intactos y funcionando (un transecto ya marcado, por ejemplo, se sigue calculando en cada análisis). Se documentan igualmente porque pueden reactivarse en cualquier momento — basta con descomentar el botón correspondiente en `App.vue` (marcado con un comentario que explica por qué está apagado) y volver a compilar.

13.4.1 Transectos — anchura de playa

Pedido en la reunión del 11/09/2026

"Olvidaros de la cuestión de los cálculos de áreas [...] como indicador interesante sí que podría ser un transecto [...] la longitud que hay desde ese punto hasta la línea que habéis segmentado. Ese indicador nos da la anchura de playa en el tramo concreto."

Sustituye al área en m^2 como indicador principal de evolución de la playa: un punto de referencia **fijo** (p. ej. la valla de la duna) + la distancia hasta la línea de costa detectada en cada análisis.

1. Ir a **Transectos** en el menú lateral y elegir la cámara.
2. Elegir una imagen de referencia (por defecto, la más reciente).
3. **Clic** sobre la imagen para añadir un transecto — aparece en la lista de la izquierda con una etiqueta editable (p. ej. "Valla duna norte").
4. *Guardar cambios*.

A partir de ahí, **cada análisis** de esa cámara (en *Resultados*, *Analizar todas* o *Modo automático*) calcula automáticamente la distancia de cada transecto hasta la línea detectada y la muestra en el panel de métricas — no hace falta volver a marcarlo.

Cómo se calcula la distancia

El backend recalcula la posición UTM del punto marcado con la homografía **vigente** en cada análisis (no la guarda fija), así que un transecto marcado hoy sigue siendo válido aunque la cámara se recalibre más adelante. La distancia es la del punto al **segmento más cercano** de la polilínea de costa — en la práctica, para un punto de referencia fijo mirando hacia el mar, equivale a la distancia perpendicular que se mediría a mano con una cinta métrica, sin necesidad de que el usuario marque también un ángulo (se discutió en la reunión y se optó por esta simplificación en vez de pedir un segundo clic de ángulo).

Un mismo transecto puede reutilizarse en todas las imágenes futuras de esa cámara — la cámara es fija (o casi, ver debate sobre viento/limpieza en el capítulo 4), así que el mismo píxel de referencia vale para cualquier imagen suya.

13.4.2 Entrenamiento línea de costa — sección temporal

Sección temporal

Pensada para que el equipo de campo marque a mano dónde está la línea de costa real en imágenes reales, y así entrenar/ajustar la segmentación automática más adelante (§13.4.3). No afecta a los resultados de *Resultados*.

Qué línea marcar: mar–arena en vez de húmedo/seco

El objetivo confirmado (04/11/2026, ver detalle en §13.4.3) es la línea que separa el **mar** de la **arena** (húmeda o seca, da igual), no la transición húmedo/seco — más contraste visual y más rápida y objetiva de marcar a mano.

1. Ir a **Entrenamiento (temp.)** en el menú lateral y elegir la cámara.
2. Elegir una imagen de la lista desplegable (o subir una propia, ver más abajo).
3. **Clic** sobre la imagen para ir añadiendo los puntos de la polilínea, en orden, siguiendo la línea mar–arena.
4. *Guardar polilínea.*

Zoom, lupa y ajuste fino

Mismo patrón que la Marcación de varillas de Calibración (capítulo 4):

- **Completa / Doble zoom**: alterna entre la imagen ajustada al panel y el doble de tamaño (con scroll).
- **Zoom de la lupa** (×2 a ×15): recorte ampliado que sigue al cursor, para acertar el píxel exacto en fotos de varios miles de píxeles de ancho.
- **Clic sobre un punto ya puesto**: lo selecciona (se pone verde y más grande) sin moverlo.
- **Flechas del teclado**, con un punto seleccionado: lo desplazan 1 px (10 px con Mayús) — para el ajuste fino que el ratón no permite con precisión.

Imágenes adicionales

No hace falta que las imágenes a marcar sean las ya cargadas de la cámara: el botón *Subir imágenes adicionales* permite traer cualquier foto suelta solo para entrenamiento — se guardan en una carpeta separada (`training_images/`) y aparecen en la lista marcadas como `[adicional]`. Se pueden eliminar de forma permanente con *Eliminar esta imagen adicional* (a diferencia de las imágenes de cámara, que solo se pueden *descartar*, ver abajo — borrarlas de verdad es cosa de *Carga de imágenes*).

Revisar y descartar antes de anotar

Pedido en la reunión del 11/09/2026

"Previsualizar, revisar y descartar imágenes antes del procesamiento [...] igual que se descartan los puntos de control."

El botón *Descartar esta imagen* saca la imagen de la cola de anotación por defecto — es un descarte **blando y reversible**: no borra el archivo, igual que descartar una varilla de calibración no borra el punto marcado, solo lo excluye del cálculo. El checkbox *Mostrar descartadas (N)* las vuelve a mostrar, con la opción de *Reincluir*.

Exportar

Exportar JSON (cámara) descarga todas las polilíneas guardadas de la cámara actual en un único fichero — pensado para compartir con el equipo (p. ej. por Google Drive) y como base para entrenar/ajustar la segmentación más adelante.

13.4.3 Lo que queda pendiente, a propósito

Cambiar el objetivo real de la segmentación de "toda la arena seca" a una línea de costa precisa es el único punto de la reunión del 11/09/2026 que se deja explícitamente para más adelante: no se puede programar a ciegas, depende de tener ya un conjunto real de imágenes anotadas (§13.4.2) con el que decidir la referencia exacta antes de tocar el algoritmo de segmentación.

Objetivo afinado (04/11/2026): línea mar–arena en vez de línea húmeda

Tras la reunión del 11/09/2026 se dejó abierto cuál era la referencia exacta (línea húmeda, marca de oleaje anterior, etc. — ver debate en el capítulo 12). El objetivo confirmado ahora es más simple y mucho menos ambiguo: la línea que separa el **mar** de la **arena** (húmeda o seca, da igual — toda la arena cuenta como una sola clase frente al agua). Es un límite con mucho más contraste visual y mucho más objetivo de marcar a mano que la transición húmedo/seco, lo que hace que anotar §13.4.2 sea más rápido y menos subjetivo. Esto no requiere ningún cambio inmediato en el SAM que usa la app hoy — sigue afinándose con las mismas herramientas (prompts por cámara, capítulo 9) — pero es el objetivo que debe seguir el equipo al marcar polilíneas nuevas, y el criterio que usará el modelo YOLO entrenado a partir de ellas (§14.6).

Capítulo 14

Instalación y Configuración

14.1 Requisitos del sistema

Cuadro 14.1: Requisitos hardware y software

Componente	Requisito mínimo
SO	Linux (Ubuntu 22.04+) o Windows 10+
Python	3.11+
RAM	8 GB (16 GB recomendado para SAM)
GPU	CUDA opcional pero recomendada para SAM
Almacenamiento	50 GB para datos (6 cámaras × 24h/día)
Node.js	18+ (para frontend)

14.2 Instalación del entorno Python

```
# venv (recomendado - el mismo que usa start_dev.ps1/start_dev.sh)
python -m venv venv
venv\Scripts\activate        # Windows
source venv/bin/activate     # Linux/macOS

pip install -r backend/requirements.txt

# Alternativa: Conda (ver environment.yml en la raíz del repositorio)
conda env create -f environment.yml
conda activate cv-lit
```

`backend/requirements.txt` instala todo lo necesario (FastAPI, OpenCV, PyTorch, `segment-anything`, `fpdf2` para los informes PDF de calibración, etc.) — no hace falta instalar paquetes sueltos a mano. El checkpoint de SAM (∼2,4 GB) no está en requirements.txt: se descarga aparte (`backend/download_sam.py`, o `download_sam.exe` en la app empaquetada) porque la app funciona sin él, con segmentación por color/Otsu de respaldo (capítulo 9).

14.3 Instalación del frontend

```
cd frontend
npm install
npm run dev       # modo desarrollo
npm run build     # build de producción (dist/)
```

14.4 Configuración de la API Obscape

Las credenciales ya NO se hardcodean en el código

Versiones anteriores de este módulo tenían el usuario y la clave de API escritos directamente en `acces_api/obscape_api.py` — se corrigió durante el endurecimiento de seguridad de esta ronda: ahora `obscape_api.py` las lee de las variables de entorno `OBSCAPE_USERNAME` y `OBSCAPE_API_KEY`, opcionalmente desde un fichero `.env` en la raíz del repositorio (`.env` está en `.gitignore` — nunca se commitea). Si faltan, `ObscapeClient()` lanza un error explícito en vez de fallar en silencio con credenciales vacías.

.env en la raíz del repositorio

```
OBSCAPE_USERNAME=<usuario del portal Obscape>
OBSCAPE_API_KEY=<clave de API>
```

Cuadro 14.2: Parámetros de conexión Obscape

Parámetro	Valor
Base URL	`https://obscape.com/portal/api/v3/api`
Username / API key	configurados por entorno, ver arriba (no se documentan aquí)

14.5 Generar el instalador de Windows

La app de escritorio se empaqueta en tres pasos, siempre en este orden — el propio `installer/cv-lit.iss` lleva esta misma receta como comentario de cabecera.

1. **Build del frontend** (genera nombres de archivo con hash de contenido nuevos en cada build):

```
cd frontend && npm run build && cd ..
```

2. **Empaquetar el backend** con PyInstaller, usando el Python del `venv` del proyecto (más fiable que depender de que `pyinstaller` esté en el PATH):

```
cd backend
```

```
..\venv\Scripts\python.exe -m PyInstaller desktop_launcher.spec --noconfirm
..\venv\Scripts\python.exe -m PyInstaller download_sam.spec --noconfirm
cd ..
```

3. **Compilar el instalador** con Inno Setup (https://jrsoftware.org/isinfo.php, instalable con `winget install -id JRSoftware.InnoSetup -e`):

```
"C:\Program Files\Inno Setup 7\ISCC.exe" cv-lit.iss
```

   El resultado queda en `installer\output\LineaDeCosta-Setup.exe` (~180 MB — el checkpoint de SAM no va incluido, ver §14).

Dos bugs reales de esta receta (corregidos el 31 de agosto de 2026)

Dos rutas del backend se calculaban con `sys._MEIPASS` cuando debían calcularse con `sys.executable` (o al revés) — el checkpoint de SAM nunca se encontraba en el `.exe` instalado, y `roi_config.json`/`alignment_masks.json` nunca se sembraban en una instalación nueva. Ambos con confianza baja como único síntoma visible, sin ningún error. El paso 2 de esta receta ya incluye el arreglo (bundle de `calibration_seed/` + `EXE_DIR` en `main.py`); el diagnóstico completo está en §12.6 (capítulo 12).

No recompilar el frontend mientras el paso 2 está en marcha

Condición de carrera real detectada en 2026-08: si se relanza `npm run build` MIENTRAS PyInstaller todavía está empaquetando (por ejemplo, en otra terminal), los nombres de archivo con hash de `frontend/dist/assets/` cambian a mitad de camino y el paquete final contiene referencias a archivos que ya no existen — la app instalada carga en blanco, sin ningún error visible durante el empaquetado (solo un aviso `WARNING: Ignoring non-existent resource` en el log de PyInstaller, fácil de pasar por alto). Si se sospecha que ha pasado, repetir el paso 2 entero sin tocar `frontend/dist/` mientras corre, y comparar que los nombres de archivo en `backend\dist\LineaDeCosta\_internal\frontend_dist\assets\` coincidan exactamente con los de `frontend/dist/assets/`.

No confiar solo en el código de salida al compilar el instalador (2026-09-23)

Bug real: tras borrar a mano `backend\build` y `backend\dist` para forzar una reconstrucción completamente limpia (sin caché de PyInstaller — necesario en esta misma sesión porque una recompilación normal del paso 2 había reutilizado un `frontend_dist` desactualizado, ver más abajo), solo se relanzó `PyInstaller desktop_launcher.spec` y no también `download_sam.spec` — al borrar la carpeta entera se había borrado también `backend\dist\download_sam.exe`, que `cv-lit.iss` necesita en su sección `[Files]`. `ISCC.exe` falló con `.Error on line 81... Source file... download_sam.exe does not exist. Compile aborted."` — pero como el propio ejecutable `ISCC.exe` se lanzó en segundo plano y solo se comprobó que la tarea había "terminado" (no el contenido real de su salida), el fallo pasó des-

apercibido y se siguió **reinstalando repetidamente un `Setup.exe` viejo, ya existente de una compilación anterior** — la instalación "funcionaba" (código de salida 0, sin ningún diálogo de error bajo `/VERYSILENT`) porque Inno Setup no borra ficheros sueltos que ya existan en el destino de una instalación previa, solo sobrescribe los que trae en su propio paquete. Sin ese paquete actualizado, nada cambiaba en el `.exe` instalado por mucho que se reinstalara. **Lección**: comprobar siempre el *texto de salida* de `ISCC.exe` buscando literalmente "`Successful compile`" (nunca solo el código de salida del proceso que lo lanzó), y verificar tras instalar que `%LOCALAPPDATA%\Programs\Linea de Costa\_internal\frontend_dist\index.html` referencia el mismo nombre de fichero `index-<hash>.js` que `frontend\dist\assets\` en ese momento — si no coincide, el instalador no copió nada nuevo de verdad, sea cual sea el código de salida que haya devuelto.

Verificación rápida tras compilar, sin necesidad de instalar nada: arrancar el ejecutable empaquetado directamente y comprobar que sirve la app entera.

```
cd backend\dist\LineaDeCosta
.\LineaDeCosta.exe
# en otra terminal:
curl http://127.0.0.1:8000/                # debe devolver el HTML del frontend
curl http://127.0.0.1:8000/api/cameras     # debe devolver JSON con las 6 cámaras
```

14.6 Entorno de desarrollo y herramientas de línea de comandos

Lo de arriba basta para generar el instalador. Esta sección es aparte, para quien va a tocar código — no hace falta nada de esto para usar la app ya instalada (capítulo 13).

Arrancar backend + frontend en modo desarrollo

```
# Windows (PowerShell) - activa venv/ y levanta backend+frontend con un
# workspace de desarrollo vacío en workspace/dev/ (no toca datos reales)
.\start_dev.ps1

# Linux/macOS (o si se prefiere Conda: ver environment.yml)
source venv/bin/activate   # o: conda activate cv-lit
./scripts/start_dev.sh

# Backend:  http://localhost:8000 (FastAPI + docs en /docs)
# Frontend: http://localhost:5173 (Vue 3 + Vite)
```

Workspace de demo - datos de muestra ya calibrados

```
.\start_demo.ps1
# Siembra workspace/demo/ desde cero cada vez (imágenes de muestra +
# varillas del GCP.csv ya importadas y calibradas). No toca los datos reales.
```

Descarga de imágenes por línea de comandos

```
# Descarga estándar (últimas 2 semanas, 12:00h)
python3 acces_api/scheduled_download.py

# Diagnóstico de estado de cámaras
python3 scripts/verify_setup.py
```

Ya no hay herramienta de calibración por línea de comandos

Las herramientas CLI que existían anteriormente (`calibration_tool.py`, `visualizar_calibracion.py`, `recalibrate.py`) se eliminaron durante el endurecimiento del proyecto para producción: quedaban duplicadas con el asistente web y sin mantenimiento. La calibración se hace exclusivamente desde la interfaz web, pestaña *Calibración* — asistente de 7 pasos, detallado en el capítulo 4. Verificar el motor matemático de la homografía se hace ahora con `python -m pytest tests/test_homography_math.py tests/test_geometry_warning.py`.

Alineación masiva por línea de comandos

```
# Alinear todas las imágenes de una carpeta respecto a la primera
python3 homography/align_images.py \
  --input  /ruta/imagenes/ \
  --output /ruta/salida/ \
  --cam 1                    # aplica máscaras SIFT de CAM_1

# Especificar imagen de referencia manualmente
python3 homography/align_images.py \
  --input  /ruta/imagenes/ \
  --output /ruta/salida/ \
  --ref    imagen_base.jpg \
  --cam 2
```

Salidas:

- `/output/aligned/` — imágenes alineadas
- `/output/diagnostics/` — visualizaciones de matches
- `/output/homographies.csv` — métricas y matrices H por imagen

Pipeline de extracción por línea de comandos

```
# Prototipo completo: ROI → Segmentación SAM → Línea de costa
python3 proces_images/test_mes3_pipeline.py --cam 1
```

Suite de tests

```
# Todo menos los tests de integración (necesitan la API real de Obscape)
python -m pytest tests/ --ignore=tests/test_api_integration.py -q
```

14.6.1 Entrenamiento de un modelo propio (YOLO-seg)

Pensado para cuando haya suficientes polilíneas reales marcadas en *Entrenamiento* (capítulo 13, §13.4.2) — primer paso hacia sustituir SAM por un modelo propio y ligero, entrenado específicamente para encontrar la línea mar–arena (§13.4.3) en vez de depender de un modelo genérico de *foundation* con prompts ajustados a mano cámara por cámara.

Dependencias aparte del resto de la app

`ultralytics` (YOLO) y `shapely` **no** están en `backend/requirements.txt` — ese fichero es justo lo que empaqueta PyInstaller dentro del `.exe`, y nada de este entrenamiento hace falta para que la app funcione en producción. Instalación aparte: `pip install -r scripts/requirements-training.txt`.

Paso 1 -- exportar las polilíneas a dataset YOLO-seg

```
python scripts/export_yolo_dataset.py --out training_dataset/mar_arena
```

Cada polilínea (línea abierta) se convierte en un polígono de entrenamiento engordándola con `shapely` un ancho fijo (`-line-width-px`, 60 px por defecto) — YOLO-seg necesita regiones cerradas, no líneas; es el mismo principio que `extract_coastline_from_mask()` pero al revés (de línea a franja, en vez de de máscara a línea). La fuente de datos es exactamente la misma que usa la pantalla *Entrenamiento* — se reutiliza la lógica real del backend (`list_training_images`, `_training_polyline_path`...) importándola de `backend/main.py`, para que las dos copias no puedan divergir con el tiempo.

Paso 2 -- entrenar

```
python scripts/train_yolo_segmentation.py --data
↪  training_dataset/mar_arena/data.yaml
```

Envoltorio fino sobre `ultralytics`: parte de `yolov8n-seg.pt` (el modelo más pequeño de la familia-seg) por defecto — con un dataset pequeño (el objetivo inicial de la reunión es ∼20 imágenes por cámara) un modelo grande sobreajusta enseguida. Los pesos entrenados quedan en `runs/segment/<nombre>/weights/best.pt`.

Verificado end-to-end con fotos reales (07/10/2026) — sin datos de producción

El pipeline completo (exportar + entrenar) se probó con 2 polilíneas de prueba sobre fotos reales (no inventadas: imágenes reales de CAM 1 y CAM 2) — se comprobó visualmente que el polígono exportado caía exactamente donde se había marcado la línea, y se lanzó un entrenamiento real de 1 época sin errores. Los datos y pesos de esa prueba se borraron después; a día de hoy no hay ninguna polilínea real guardada en producción — hace falta que el equipo marque las primeras antes de entrenar un modelo de verdad.

Apéndice A

Convenciones de código y Git

A.1 Estrategia de ramas

- **main**: código estable y validado.
- **feature/nombre-modulo**: nuevas funcionalidades.
- **fix/descripcion-error**: correcciones.
- **docs/nombre-doc**: documentación.

Todo cambio debe pasar por Pull Request antes de integrar en `main`.

A.2 Nomenclatura de ficheros

Las imágenes descargadas usan el formato determinista:

`UNIX_YYYYMMDD_HHMMSS_REFERENCIA.jpg`

A.3 Proyección obligatoria

Toda coordenada geográfica en el sistema debe estar en EPSG:25830. Nunca almacenar en WGS84 (EPSG:4326) salvo para visualización en APIs de mapas.

Apéndice B

Glosario

EPSG:25830 ETRS89 / UTM zona 30N. Sistema de referencia cartográfico oficial para la Península Ibérica.

GCP *Ground Control Point*. Punto de control terrestre con coordenadas conocidas.

Homografía Transformación proyectiva planar entre dos planos (imagen y terreno).

IEL Instituto de Ecología Litoral. Suministra los GCPs GNSS del proyecto.

RANSAC *Random Sample Consensus*. Estimador robusto de parámetros geométricos.

ROI *Region of Interest*. Zona de la imagen relevante para el análisis.

SAM *Segment Anything Model*. Modelo de segmentación de Meta AI.

SIFT *Scale-Invariant Feature Transform*. Detector de puntos de interés invariante a escala y rotación.

RMSE *Root Mean Square Error*. Error cuadrático medio.

MAE *Mean Absolute Error*. Error medio absoluto.

Farneback Algoritmo de flujo óptico denso para estimar campo de desplazamiento entre dos imágenes.

Inliers Correspondencias que se ajustan al modelo estimado por RANSAC.

Two-phase commit Patrón de escritura en dos fases: staging temporal → confirmación permanente.

Drift Desplazamiento residual de píxeles tras la corrección de homografía.

Auto Mode Módulo que automatiza, para un rango de fechas elegido, la secuencia descarga → alineación → análisis sobre una cámara ya calibrada (capítulo 11).

Aviso de geometría inestable (`geometry_warning`) Mensaje que advierte de que RANSAC no encontró un subconjunto mutuamente consistente de varillas suficientemente grande al calcular la homografía — el RMSE resultante puede no reflejar la calidad real de la calibración (capítulo 4).

Varillas usadas

Número de GCPs que entran en el ajuste de la homografía (confirmadas y no excluidas) — **no** es lo mismo que *inliers*, que es cuántas de esas RANSAC clasifica como mutuamente consistentes (capítulo 4).

Apéndice C

Historial de cambios

Cuadro C.1: Historial de versiones del sistema

Fecha	Componente	Cambio
Ene 2026	API client	Cliente Obscape + descarga con deduplicación
Feb 2026	Calibración	`calibration_tool.py`: intrínsecos + validación ciega
Mar 2026	Calibración	`recalibrate.py`: recalibración por arrastre de GCPs
Abr 2026	Segmentación	Prototipo SAM + extracción de línea de costa
May 2026	Georreferenciación	Proyección a EPSG:25830 + exportación GeoJSON
May 2026	Calibración	Ejecución real con GCPs del IEL (6 cámaras)
Jun 2026	Backend	`batch_alignment.py`: pipeline asíncrono + 10 endpoints
Jun 2026	Frontend	`BatchAlignment.vue`: máquina de estados 5 fases
Jun 2026	Máscaras SIFT	`alignment_masks.json`: zonas estables por cámara
Jun 2026	CAM 1	Corrección máscara: excluir cielo con gaviotas (y=0.24→0.31)
Jun 2026	Documentación	Esta memoria técnica en LaTeX
Jul 2026	Backend	`auto_mode.py`: pipeline automático descarga+alineación+análisis
Jul 2026	Frontend	`AutoMode.vue`: asistente de cámara + rango de fechas
Jul–Ago 2026	Seguridad	CORS restringido, validación de `PUT`, credenciales de Obscape movidas a variables de entorno, eliminación de scripts sin mantenimiento (`calibration_tool.py`, `recalibrate.py` y otros)

(continúa en la página siguiente)

(continúa de la página anterior)

Fecha	Componente	Cambio
Ago 2026	Calibración	Aviso de geometría inestable (`geometry_warning`), inliers agregados, histograma de residuos y mapa de cobertura en Validación
Ago 2026	Calibración	Metadatos de perfil + exportación de informe JSON/PDF por cámara
Ago 2026	Marcación	Cota Z opcional, zoom ajustable de la lupa, barra de progreso
Ago 2026	Auto Mode	Exportación automática de GeoJSON, panel de estado por cámara, log filtrado en tiempo real
Ago 2026	Empaquetado	Instalador de escritorio para Windows (`LineaDeCosta-Setup.exe`, PyInstaller + Inno Setup)
Ago 2026	Tests	`test_auto_mode.py`, `test_geometry_warning.py`
Sep 2026	Resultados	Caché de resultado por imagen (flechas de navegación ya no relanzan SAM) + `_SAM_LOCK`: corrige un fallo real de concurrencia (`SamPredictor` no es reentrante)
Sep 2026	Segmentación	`fill_small_gaps()` reescrito con reconstrucción morfológica; prompts de CAM 2/3 ajustados; recorte universal de la franja gris de fecha/hora
Sep 2026	Entrenamiento	Sección temporal nueva (`TrainingAnnotation.vue`): marcado manual de la línea húmeda por polilíneas, zoom/lupa/flechas de precisión, imágenes adicionales, revisar/descartar, exportación JSON por cámara
Sep 2026	Transectos	Pantalla nueva (`Transects.vue`) + `_compute_transect_ distances()`: anchura de playa por punto de referencia fijo, sustituye al área en fiabilidad (pedido en reunión de equipo, 11/09/2026)
Sep 2026	Resultados	Edición manual de la línea detectada por SAM (arrastrar/añadir/borrar puntos), guardada como dato de entrenamiento en el mismo almacén que `TrainingAnnotation.vue`

(continúa en la página siguiente)

(continúa de la página anterior)

Fecha	Componente	Cambio
Sep 2026	Carga de imágenes	Visor de previsualización a tamaño completo antes de procesar/descartar una imagen
Oct 2026	Segmentación	`scripts/export_yolo_dataset.py` + `train_yolo_ segmentation.py`: exporta las polilíneas de Entrenamiento a dataset YOLO-seg (línea → franja vía `shapely.buffer`) y entrena un YOLOv8n-seg — primer paso hacia sustituir SAM por un modelo propio de línea de costa (§13.4.3)
Nov 2026	Proyecto	El programa de sensorización de Guardamar recibe nombre propio: **SensoMar**
Nov 2026	Cámaras	Las 6 cámaras pasan a capturar en ráfaga (1 Hz, 10 fotos) a las 12:00 y 13:00; sin cambios de código — `download_image_flat()` ya nombra cada fichero con segundos (`<epoch>_<AAAAMMDD>_<HHMMSS>_<serie>.jpg`), así que las 10 fotos de cada ráfaga no colisionan
Nov 2026	Segmentación	Objetivo de anotación afinado: la referencia a marcar ya no es la línea húmeda (ambigua) sino la línea que separa el **mar** de la **arena** (§13.4.3) — más objetiva y rápida de marcar a mano